

Pemanfaatan Algoritma Greedy dalam pembuatan *bot* permainan Robocode Tank Royale

Tugas Besar

Diajukan sebagai syarat menyelesaikan mata kuliah Strategi Algoritma Kelas RA
di Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi
Sumatera



Oleh: Kelompok 4 (Ayam Ga Jago)

Guntur Anggoro Rahardianto 124140105

Muhammad Radhitya 124140189

Dina Olivia 124140213

Dosen Pengampu: **Imam Eko Wicaksono, S.Si., M.Si.**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA**

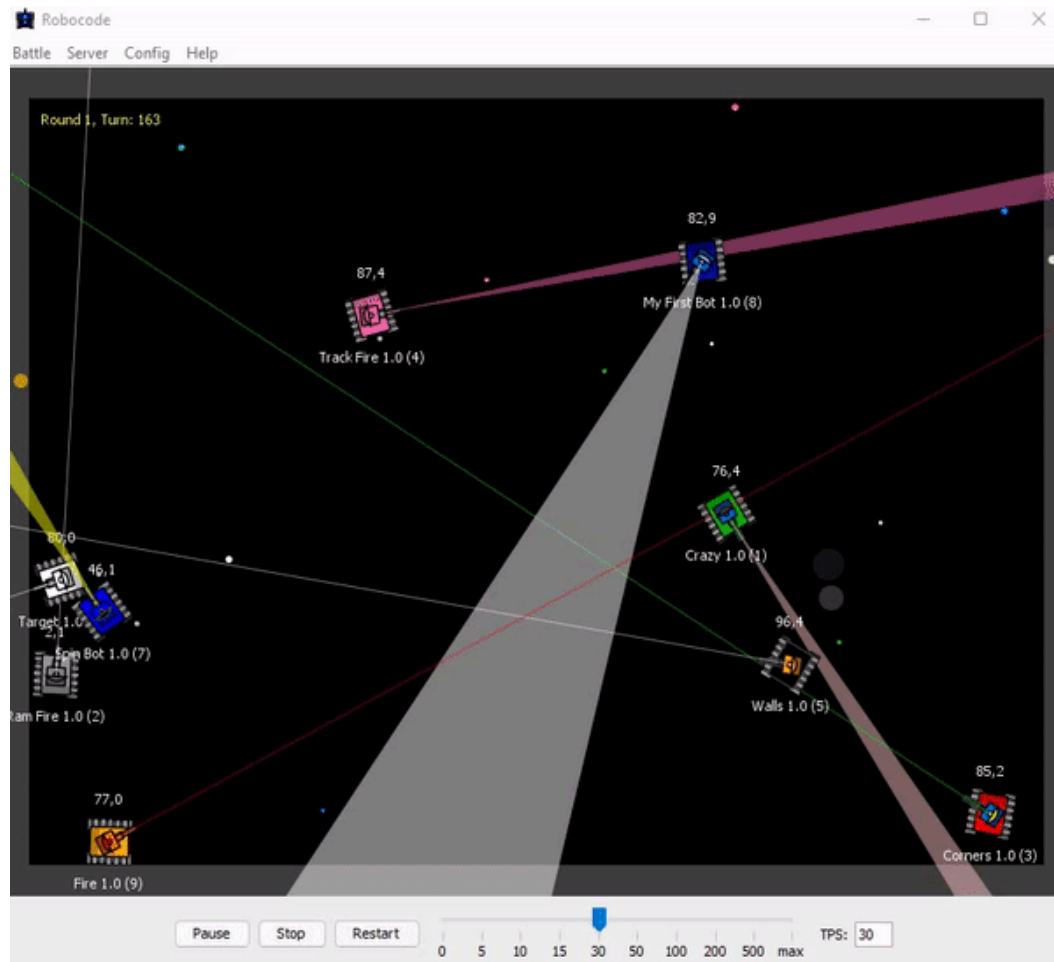
2026

Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Robocode Tank Royale.....	1
BAB I DESKRIPSI TUGAS.....	4
1.1 Latar Belakang.....	5
1.2 Tujuan Tugas Besar.....	5
1.3 Ruang Lingkup Tugas.....	6
1.4 Spesifikasi Tugas.....	6
Spesifikasi Wajib.....	7
Spesifikasi Bonus.....	7
BAB II.....	10
LANDASAN.....	10
TEORI.....	10
2.1 Dasar Teori.....	10
2.1.1 Algoritma Greedy.....	10
2.1.2 Mekanisme Permainan Robocode Tank Royal.....	11
2.1.3 Sistem Energi dan Skor.....	11
2.1.4 Sistem Radar dan Deteksi Musuh.....	12
2.1.5 Strategi Penembakan dan Prediksi Posisi Target.....	13
2.1.6 Strategi Pergerakan (Movement).....	13
2.1.7 Sistem Multi-Agen dan Pemilihan Target.....	14
2.1.8 Implementasi Bot dalam C# (.NET).....	15
2.2 Cara Kerja Program.....	15
BAB III.....	17
3.1 Proses Mapping Persoalan ke Elemen Greedy.....	17
3.1.1 Himpunan Kandidat.....	17
3.1.2 Himpunan Solusi.....	17
3.1.3 Himpunan Seleksi.....	18
3.1.4 Fungsi Kelayakan.....	18
3.1.4 Fungsi Objektif.....	18
3.2 Proses Mapping Persoalan ke Elemen Greedy.....	19
3.2.1 Bot Utama – TankAlbino.....	19
3.2.2 Bot Alternatif 1 – BantengMerah.....	21
3.2.3 Bot Alternatif 2 – AiranScube.....	22
3.2.4 Bot Alternatif 3 – Gryffin.....	23
3.3 Analisis Efisiensi dan Efektivitas.....	25
3.4 Strategi Greedy yang Dipilih.....	25
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	27
4.1 Implementasi Algoritma Greedy.....	27
4.1.1 Pseudocode Main-bot.....	27
4.1.2 Pseudocode Alternative Bot 1.....	42
4.1.3 Pseudocode Alternative Bot 2.....	53
4.1.4 Pseudocode Alternative Bot 3.....	69
4.2 Struktur Data, Fungsi, dan Prosedur Bot Utama.....	92
4.2.1. Struktur Data (Penyimpanan Informasi).....	92
4.2.2. FUNGSI (Kalkulasi & Menghasilkan Nilai).....	93

4.2.3. PROSEDUR (Eksekusi Aksi & Merubah State).....	93
4.3 Pengujian Program.....	95
4.3.1 Skenario Pengujian.....	95
4.3.2 Hasil Pengujian dan Analisis.....	97
BAB V KESIMPULAN DAN SARAN.....	100
5.1 Kesimpulan.....	100
5.2 Saran.....	101
LAMPIRAN.....	102
DAFTAR PUSTAKA.....	103

BAB I

DESKRIPSI TUGAS



Gambar 1. Robocode Tank Royale

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

1.1 Latar Belakang

Perkembangan teknologi dan kecerdasan buatan (Artificial Intelligence/AI) mendorong munculnya berbagai media pembelajaran interaktif, salah satunya melalui permainan berbasis pemrograman. Permainan ini tidak hanya digunakan sebagai hiburan, tetapi juga sebagai sarana untuk melatih kemampuan logika, analisis, dan penyusunan strategi algoritma. Salah satu permainan yang mendukung hal tersebut adalah Robocode Tank Royale.

Robocode Tank Royale merupakan permainan pemrograman di mana pemain membuat bot berbentuk tank virtual untuk bertarung melawan bot lain di arena. Pemain tidak mengendalikan tank secara langsung, melainkan membuat program yang menentukan cara bot bergerak, memindai musuh, menyerang, dan bertahan selama pertandingan berlangsung. Pertarungan berlangsung hingga hanya tersisa satu bot sebagai pemenang sehingga permainan ini memiliki konsep seperti Battle Royale.

Dalam permainan ini terdapat berbagai mekanisme seperti energi, radar, peluru, pergerakan, dan sistem skor yang membuat penyusunan strategi menjadi penting. Oleh karena itu, Robocode Tank Royale dapat dijadikan media yang tepat untuk menerapkan algoritma greedy. Algoritma greedy bekerja dengan memilih keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu untuk memperoleh hasil yang optimal.

Pada tugas besar ini, algoritma greedy diimplementasikan pada bot Robocode Tank Royale menggunakan bahasa pemrograman C# berbasis .NET. Setiap bot dirancang dengan strategi dan heuristic yang berbeda untuk memperoleh skor setinggi mungkin dalam pertandingan. Melalui tugas ini, diharapkan mahasiswa dapat memahami penerapan algoritma greedy dalam permasalahan nyata sekaligus meningkatkan kemampuan pemrograman dan penyusunan strategi.

1.2 Tujuan Tugas Besar

Tujuan utama dari Tugas Besar ini adalah sebagai berikut:

1. Menerapkan konsep algoritma greedy untuk merancang dan mengimplementasikan strategi pada sebuah agen (bot) dalam permainan "Robocode Tank Royale".

2. Mengembangkan program bot menggunakan bahasa pemrograman C# berbasis .NET yang mampu berinteraksi dengan game engine serta mengambil keputusan secara otomatis berdasarkan strategi greedy yang dirancang untuk memperoleh skor setinggi mungkin.
3. Menganalisis dan membandingkan beberapa alternatif strategi greedy beserta heuristic yang berbeda untuk menentukan solusi yang paling efektif dalam permainan Robocode Tank Royale
4. Mendokumentasikan proses perancangan, implementasi, pengujian, serta analisis algoritma greedy ke dalam laporan ilmiah sesuai dengan ketentuan tugas besar.
5. mengkompetisikan bot yang telah dikembangkan dengan bot dari kelompok lain untuk mengevaluasi performa dan efektivitas strategi greedy yang diterapkan.

1.3 Ruang Lingkup Tugas

Ruang lingkup pengerjaan Tugas Besar ini meliputi:

1. Memahami aturan permainan Robocode Tank Royale, mekanisme pertandingan, sistem skor, serta interaksi antara bot dan game engine.
2. Menggunakan game engine dan starter pack yang telah disediakan untuk menjalankan serta mengembangkan bot permainan.
3. Merancang dan mengembangkan 4 bot (1 bot utama dan 3 bot alternatif) menggunakan bahasa pemrograman C# berbasis .NET dengan menerapkan strategi algoritma greedy yang berbeda.
4. Merancang heuristic dan strategi greedy yang mempertimbangkan berbagai aspek permainan, seperti pergerakan bot, pemindaian radar, penggunaan energi, pemilihan target, penembakan, serta penghindaran serangan musuh.
5. Mengimplementasikan strategi greedy ke dalam logika bot agar mampu mengambil keputusan secara otomatis selama pertandingan berlangsung.
6. Melakukan pengujian dan analisis performa bot melalui beberapa simulasi pertandingan untuk mengevaluasi efektivitas strategi yang diterapkan
7. Menyusun laporan tugas besar yang mencakup deskripsi tugas, landasan teori, perancangan algoritma greedy, implementasi program, hasil pengujian, serta analisis performa bot.

Tugas ini tidak mencakup modifikasi terhadap game engine yang telah disediakan. Fokus utama tugas berada pada perancangan strategi greedy dan implementasi logika bot

1.4 Spesifikasi Tugas

Berikut adalah spesifikasi rinci terkait pelaksanaan dan penilaian Tugas Besar ini:

Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif) dalam bahasa C# ([.net](#)) yang mengimplementasikan *algoritma Greedy* pada *bot* permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**, boleh lintas kelas dan lintas kampus.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan **heuristic** yang berbeda.
- **Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.**
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes 1.
- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada game engine** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca [dokumentasi](#) dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian [starter pack](#)

Spesifikasi Bonus

- (maks 10) Membuat video tentang aplikasi *greedy* pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan

menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. **Semakin menarik video, maka semakin banyak poin yang diberikan.**

- (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.
- Jika terdapat kesulitan selama mengerjakan tugas besar sehingga memerlukan bimbingan, maka dapat melakukan asistensi tugas besar kepada asisten (opsional). Dengan catatan asistensi hanya bersifat membimbing, bukan memberikan “jawaban”.
- Bot yang telah dibuat akan dikompetisikan dengan kelompok lain dan disaksikan oleh seluruh peserta kuliah. Terdapat hadiah menarik bagi kelompok yang memenangkan kompetisi.
- Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, paling lambat **Kamis, 6 Maret pukul 22.11 WIB**.

[📄 Pendataan Kelompok Tubes 1 Stima 2024/2025](#)

- Diwajibkan untuk memilih asisten meskipun tidak melakukan asistensi, karena asisten yang dipilih akan menjadi asisten saat asistensi (opsional) dan demo tugas besar. Pemilihan asisten dapat dilakukan pada link berikut, paling lambat **Kamis, 6 Maret 2025 pukul 22.11 WIB**.

[📄 Pendataan Kelompok Tubes 1 Stima 2024/2025](#)

- Program disimpan dalam repository yang bernama **Tubes1_NamaKelompok** dengan nama kelompok sesuai dengan yang di sheets diatas. Berikut merupakan struktur dari isi repository tersebut:

```
|— src/
|   |— main-bot/...
|   |— alternative-bots/
|       |— alt-bot-1/...
|       |— alt-bot-2/...
|       |— alt-bot-3/...
|— doc/
|   |— laporan.pdf
|— README.md
```


- a. Folder src berisi semua source code
 - b. Folder doc berisi laporan tugas besar dengan format **NamaKelompok.pdf** dengan ketentuan isi pada [isi laporan](#)
 - c. README untuk tata cara penggunaan yang minimal berisi:
 - i. Penjelasan singkat algoritma greedy yang diimplementasikan untuk setiap bot yang dibuat
 - ii. Requirement program dan instalasi tertentu bila ada
 - iii. Command atau langkah-langkah dalam meng-compile atau build program
 - iv. Author (identitas pembuat)
- Sangat disarankan untuk menggunakan [semantic commit](#). Buatlah release dengan format **v1.x** dengan x adalah nomor revisi dimulai dari revisi 0. Contoh v1.0 untuk release pertama, v1.1 untuk revisi selanjutnya.
 - Pastikan untuk membuat repository bersifat **Public** paling lambat **H+1 deadline (1 hari setelah deadline)**. Sebelum deadline repository harus bersifat **Private**.
 - Laporan dikumpulkan hari **Senin, 24 Maret 2025** pada alamat Google Form berikut paling lambat **pukul 22.11 WIB**:
<https://bit.ly/tubes1stima25>.
- PERINGATAN: Keterlambatan akan mengurangi nilai sebanyak 1 poin untuk setiap menit keterlambatan.**
- Adapun pertanyaan terkait tugas besar ini bisa disampaikan melalui QnA berikut:
<https://bit.ly/QnA-Stima-25>.

BAB II

LANDASAN

TEORI

2.1 Dasar Teori

2.1.1 Algoritma Greedy

Algoritma greedy adalah salah satu paradigma perancangan algoritma yang populer untuk menyelesaikan masalah optimasi. Prinsip dasar dari algoritma greedy adalah membuat serangkaian pilihan yang tampak optimal secara lokal pada setiap langkah dengan harapan akan mengarah pada solusi optimal global. Pada setiap tahap pengambilan keputusan, algoritma greedy memilih opsi yang memberikan keuntungan terbesar atau biaya terkecil saat itu, tanpa mempertimbangkan konsekuensi pilihan tersebut terhadap keputusan di masa depan.

Dalam konteks permainan Robocode Tank Royale, algoritma greedy diaplikasikan untuk setiap keputusan taktis yang diambil bot pada setiap giliran (turn). Bot akan membuat keputusan berdasarkan kondisi permainan saat itu, misalnya memilih target dengan energi paling rendah untuk diserang, menentukan daya tembak peluru yang optimal, atau memilih arah pergerakan yang paling aman. Setiap keputusan lokal ini bertujuan memaksimalkan skor akhir bot di penghujung pertandingan.

Karakteristik utama dari algoritma greedy meliputi:

1. **Prinsip Greedy Choice Property:** Solusi optimal global dapat dicapai dengan membuat pilihan optimal secara lokal. Pilihan yang dibuat pada setiap langkah tidak bergantung pada pilihan sebelumnya atau pilihan di masa depan, melainkan hanya pada kondisi saat itu.
2. **Optimal Substructure:** Solusi optimal untuk suatu masalah mengandung solusi optimal untuk sub-masalahnya. Jika sebuah pilihan optimal lokal dibuat, masalah yang tersisa juga harus memiliki solusi optimal setelah pilihan tersebut diterapkan.
3. **Efisiensi Komputasi:** Algoritma greedy umumnya memiliki kompleksitas waktu yang lebih rendah dibandingkan pendekatan dinamis atau exhaustive search, sehingga sangat cocok untuk lingkungan real-time seperti Robocode di mana keputusan harus diambil dalam setiap tick permainan.

Meskipun algoritma greedy seringkali intuitif dan mudah diimplementasikan, tidak semua masalah optimasi dapat diselesaikan secara optimal menggunakan pendekatan ini.

Keberhasilan algoritma greedy sangat bergantung pada struktur masalah yang dihadapi. Beberapa masalah klasik yang dapat diselesaikan secara optimal dengan algoritma greedy antara lain adalah algoritma Prim dan Kruskal untuk mencari Minimum Spanning Tree (MST), algoritma Dijkstra untuk mencari lintasan terpendek, serta masalah penukaran koin.

2.1.2 Mekanisme Permainan Robocode Tank Royal

Robocode Tank Royale adalah platform permainan pemrograman di mana setiap pemain membuat bot berupa tank virtual yang bertempur satu sama lain dalam sebuah arena. Berbeda dari permainan konvensional, pemain tidak mengendalikan tank secara langsung, melainkan menulis kode program yang menentukan logika dan perilaku bot secara otomatis. Pertempuran berlangsung hingga hanya tersisa satu bot, menjadikannya format Battle Royale.

Setiap bot dalam Robocode Tank Royale terdiri dari tiga komponen utama yang bergerak secara independen:

- **Badan Tank (Body):** Komponen utama yang bergerak maju, mundur, dan berbelok di arena. Pergerakan body mempengaruhi arah keseluruhan bot.
- **Turret (Gun):** Laras senjata yang dapat berputar independen dari body. Turret menembakkan peluru ke arah yang ditentukan. Turret memiliki heat (panas) setelah menembak yang harus mendingin sebelum bisa menembak lagi.
- **Radar:** Sensor berbentuk kipas/cone yang dapat berputar independen dari body maupun gun. Radar berfungsi mendeteksi keberadaan bot lawan. Bot hanya bisa mengetahui posisi musuh jika radar sedang menyapu posisi musuh tersebut.

Perputaran ketiga komponen ini memiliki batasan kecepatan per tick. Body dapat berputar maksimal $10^\circ/\text{tick}$ (dikurangi $0,75^\circ$ per unit kecepatan), gun dapat berputar maksimal $20^\circ/\text{tick}$, dan radar dapat berputar maksimal $45^\circ/\text{tick}$. Pemahaman atas batasan perputaran ini sangat penting dalam merancang strategi greedy yang efektif.

2.1.3 Sistem Energi dan Skor

Energi adalah sumber daya utama setiap bot dalam Robocode Tank Royale. Setiap bot dimulai dengan energi penuh (umumnya 100 unit). Energi berkurang ketika bot terkena peluru atau menabrak dinding/bot lain, dan bertambah ketika peluru yang ditembakkan bot tersebut mengenai musuh. Ketika energi bot mencapai nol, bot tersebut hancur (disabled) dan keluar dari pertandingan.

Mekanisme energi yang relevan untuk strategi greedy adalah sebagai berikut:

- **Biaya Menembak:** Menembak peluru dengan daya (firepower) sebesar p akan mengurangi energi sebesar p . Nilai p berkisar antara 0,1 hingga 3. Semakin besar daya tembak, semakin besar kerusakan yang diberikan ke musuh, tetapi semakin besar pula energi yang dikonsumsi.
- **Kerusakan Peluru:** Peluru dengan daya p menyebabkan kerusakan sebesar $4 \times p$ pada musuh. Jika daya p lebih besar dari 1, maka kerusakan tambahan sebesar $2 \times (p-1)$ diberikan, sehingga total kerusakan menjadi $4p + 2(p-1) = 6p - 2$ untuk $p > 1$.
- **Pemulihan Energi:** Ketika peluru mengenai musuh, penembak memperoleh kembali energi sebesar $3 \times p$. Mekanisme ini menciptakan insentif greedy untuk aktif menyerang.
- **Penalti Tabrakan:** Menabrak dinding arena mengurangi energi bot. Menabrak bot lain juga memberikan penalti energi pada kedua pihak. Karena itu, manuver penghindaran adalah bagian penting dari strategi greedy.

Sistem skor dalam Robocode Tank Royale mempertimbangkan beberapa faktor: total kerusakan yang diberikan ke musuh (damage score), bonus untuk setiap kill, bonus bertahan hidup (survival), dan bonus menjadi pemenang terakhir (last survivor bonus). Memahami komponen skor ini adalah kunci dalam merancang fungsi objektif untuk algoritma greedy, yakni memaksimalkan total skor di akhir pertandingan.

2.1.4 Sistem Radar dan Deteksi Musuh

Radar adalah komponen kritis dalam Robocode Tank Royale karena bot hanya dapat memperoleh informasi tentang posisi musuh melalui sapuan radar. Ketika radar menyapu posisi sebuah bot lawan, event ScannedBot akan terpicu, memberikan informasi berupa posisi (x, y), energi, kecepatan, dan arah pergerakan musuh pada saat itu. Beberapa strategi penggunaan radar yang relevan dalam konteks greedy adalah:

- **Infinite Radar Lock (Radar Kunci Tak Terbatas):** Teknik di mana radar terus menerus mengikuti satu musuh dengan cara membalikkan arah putaran radar sedikit melewati posisi musuh setiap kali musuh terdeteksi. Teknik ini memastikan informasi posisi musuh selalu terkini.
- **Wide Radar Sweep:** Radar diputar penuh untuk memindai seluruh arena. Teknik ini berguna saat bot ingin memilih target secara greedy berdasarkan kriteria terbaik (energi terendah, jarak terdekat) dari semua musuh yang ada.
- **Narrow Radar Lock:** Radar diputar minimal untuk tetap mengunci satu musuh. Teknik ini mengoptimalkan frekuensi pembaruan data musuh dan meminimalkan sumber daya

putaran radar.

Dalam konteks algoritma greedy, pemilihan strategi radar merupakan keputusan greedy tersendiri: apakah lebih menguntungkan untuk mengunci satu musuh demi akurasi tembakan yang lebih tinggi, atau menyapu seluruh arena demi pemilihan target yang lebih optimal secara global. Pilihan ini mempengaruhi efektivitas keseluruhan strategi bot.

2.1.5 Strategi Penembakan dan Prediksi Posisi Target

Karena peluru dalam Robocode Tank Royale memiliki kecepatan terbatas ($20 - 3 \times \text{firepower unit per tick}$), bot tidak dapat langsung menembak ke posisi musuh saat ini. Sebaliknya, bot harus memprediksi posisi musuh di masa depan berdasarkan data terakhir dari radar, termasuk posisi, kecepatan, dan arah pergerakan musuh.

Beberapa metode penembakan yang relevan dengan pendekatan greedy adalah:

- **Linear Targeting (Penembakan Linear):** Mengasumsikan musuh akan terus bergerak lurus dengan kecepatan dan arah konstan. Bot menghitung posisi musuh setelah t tick (waktu tempuh peluru) dan mengarahkan tembakan ke posisi tersebut. Ini adalah pendekatan greedy sederhana yang efektif terhadap musuh bergerak lurus.
- **Head-On Targeting:** Menembak langsung ke posisi musuh saat ini tanpa memperhitungkan pergerakan. Meskipun akurasi rendah, metode ini sangat sederhana dan cukup efektif jika musuh bergerak lambat atau diam.
- **Circular Targeting:** Mengasumsikan musuh bergerak dalam pola melingkar (berbelok dengan kecepatan angular konstan). Metode ini lebih akurat untuk musuh yang menerapkan gerakan berputar.

Pemilihan daya tembak (firepower) juga merupakan keputusan greedy penting. Daya tembak tinggi memberikan kerusakan lebih besar dan pemulihan energi lebih banyak, tetapi mengonsumsi energi lebih besar dan menghasilkan peluru yang lebih lambat (sehingga lebih mudah dihindari musuh). Daya tembak rendah menghasilkan peluru lebih cepat dengan konsumsi energi minim, tetapi kerusakan yang diberikan lebih kecil. Strategi greedy yang efektif mempertimbangkan jarak ke musuh dan kondisi energi kedua pihak dalam memilih firepower optimal.

2.1.6 Strategi Pergerakan (Movement)

Pergerakan bot dalam Robocode Tank Royale bertujuan ganda: mendekatkan diri ke posisi yang menguntungkan untuk menembak sekaligus menghindari peluru dari musuh. Kecepatan maksimal bot adalah 8 unit/tick, dengan akselerasi/deselerasi sebesar 1 unit/tick. Strategi pergerakan merupakan salah satu komponen terpenting dalam merancang bot yang kompetitif.

Beberapa pendekatan pergerakan yang umum digunakan dalam konteks greedy adalah:

- **Perpendicular Movement (Pergerakan Tegak Lurus):** Bot bergerak tegak lurus terhadap arah tembakan musuh. Karena peluru bergerak lurus, gerakan tegak lurus memaksimalkan jarak yang ditempuh peluru musuh sebelum mengenai bot. Ini adalah salah satu teknik penghindaran paling efektif.
- **Oscillating Movement (Pergerakan Osilasi):** Bot bergerak maju-mundur secara periodik sambil mempertahankan orientasi tegak lurus terhadap musuh. Variasi arah yang tidak dapat diprediksi ini mempersulit musuh dalam menerapkan linear targeting.
- **Wall Avoidance (Penghindaran Dinding):** Bot mendeteksi kedekatan dengan dinding arena dan mengubah arah untuk menghindari tabrakan yang menyebabkan penalti energi. Dalam keputusan greedy, penghindaran dinding diprioritaskan bila jarak ke dinding di bawah ambang batas tertentu.
- **Adaptive Distance Keeping:** Bot mempertahankan jarak optimal terhadap musuh—tidak terlalu dekat (risiko tabrakan) dan tidak terlalu jauh (akurasi tembakan menurun). Algoritma greedy memilih langkah pergerakan yang mempertahankan jarak ini secara dinamis.

2.1.7 Sistem Multi-Agen dan Pemilihan Target

Dalam pertandingan Robocode Tank Royale dengan lebih dari dua bot (format Battle Royale), setiap bot beroperasi dalam lingkungan multi-agen (Multi-Agent System/MAS) di mana terdapat banyak agen otonom yang berinteraksi sekaligus. Kondisi ini menambah dimensi kompleksitas karena aksi bot lain dapat mempengaruhi kondisi permainan secara signifikan.

Konsep multi-agen yang relevan dalam konteks greedy Robocode meliputi:

- **Pemilihan Target Greedy:** Dalam format multi-bot, bot harus memilih satu target dari beberapa musuh yang terdeteksi. Pendekatan greedy memilih target berdasarkan heuristik tertentu, seperti musuh dengan energi terendah (kemungkinan kill tertinggi), musuh terdekat (akurasi tembakan terbaik), atau musuh yang sedang mengincar bot kita (ancaman tertinggi).
- **Strategi Memanfaatkan Pertempuran Musuh:** Bot dapat secara greedy memilih untuk tidak langsung terlibat pertempuran dan membiarkan musuh-musuh lain saling melemahkan, lalu menyerang musuh yang sudah lemah. Strategi ini memaksimalkan efisiensi energi bot.
- **Posisi Optimal di Arena:** Dalam skenario multi-bot, posisi tengah arena umumnya memberi visibilitas radar terbaik dan jarak seimbang ke semua musuh. Keputusan greedy untuk memilih posisi mempertimbangkan ancaman dari berbagai arah sekaligus.

Tantangan utama dalam lingkungan multi-agen adalah ketidakpastian aksi musuh. Algoritma greedy mengatasi hal ini dengan membuat keputusan terbaik berdasarkan informasi yang tersedia

pada tick saat itu, dan memperbarui keputusan tersebut di setiap tick berikutnya seiring informasi baru diperoleh dari radar.

2.1.8 Implementasi Bot dalam C# (.NET)

Robocode Tank Royale mendukung pembuatan bot dalam berbagai bahasa pemrograman, termasuk C# berbasis .NET. Framework C# dipilih dalam tugas ini karena kemampuannya dalam pemrograman berorientasi objek, performa tinggi, dan dukungan ekosistem .NET yang lengkap. Setiap bot C# ditulis sebagai kelas yang mewarisi kelas dasar Bot dari library Robocode.TankRoyale.BotApi.

Struktur utama bot C# dalam Robocode Tank Royale terdiri dari beberapa komponen:

- **Metode Run():** Loop utama bot yang berjalan terus-menerus selama pertandingan. Di dalam loop ini, bot mengeluarkan perintah pergerakan, penembakan, dan rotasi radar secara berulang.
- **Event Handler OnScannedBot():** Dipanggil setiap kali radar mendeteksi musuh. Di sinilah sebagian besar logika greedy untuk targetting dan penembakan diimplementasikan berdasarkan data musuh yang baru diterima.
- **Event Handler OnHitByBullet():** Dipanggil saat bot terkena peluru. Dapat digunakan untuk mengaktifkan manuver penghindaran sebagai respons greedy terhadap ancaman.
- **Event Handler OnHitWall():** Dipanggil saat bot menabrak dinding. Digunakan untuk mengoreksi pergerakan dan menghindari penalti energi lebih lanjut.
- **Event Handler OnBotDeath():** Dipanggil saat bot lain hancur. Dapat digunakan untuk memperbarui strategi greedy, misalnya memilih target baru atau mengubah mode permainan.

Setiap perintah yang diberikan ke bot (seperti SetForward(), SetTurnLeft(), SetFirepower(), SetTurnGunLeft(), SetTurnRadarLeft()) bersifat asinkron dan dieksekusi secara paralel dalam satu tick. Pemahaman atas model eksekusi ini penting agar keputusan greedy dapat diimplementasikan dengan tepat dan efisien dalam kode C#.

2.2 Cara Kerja Program

Secara umum, bot Robocode Tank Royale yang mengimplementasikan algoritma greedy bekerja dalam siklus per tick sebagai berikut. Pertama, pada setiap tick, bot membaca status permainan saat ini melalui event dan properti yang disediakan oleh game engine. Informasi ini mencakup posisi dan energi bot sendiri, data musuh terakhir dari radar, serta kondisi arena (dinding, posisi).

Kedua, berdasarkan informasi tersebut, bot mengevaluasi kondisi menggunakan heuristik greedy yang telah dirancang untuk menentukan aksi terbaik pada tick ini. Evaluasi dilakukan secara hierarkis: prioritas keselamatan (menghindari dinding dan tabrakan) diperiksa terlebih dahulu, diikuti prioritas pertempuran (pemilihan target, kalkulasi firepower, arah tembakan), kemudian pergerakan (perpendicular movement, osilasi).

Ketiga, perintah yang telah diputuskan dikirimkan ke game engine untuk dieksekusi pada tick tersebut. Proses ini berulang setiap tick selama pertandingan berlangsung, memungkinkan bot beradaptasi secara real-time terhadap perubahan kondisi permainan. Penjelasan lebih rinci mengenai implementasi masing-masing komponen dipaparkan pada bab-bab selanjutnya.

BAB III

APLIKASI STRATEGI *GREEDY*

3.1 Proses Mapping Persoalan ke Elemen Greedy

Permainan Robocode Tank Royale merupakan permainan strategi real-time di mana bot harus mengambil keputusan secara cepat berdasarkan kondisi arena yang terus berubah. Pada setiap giliran permainan, bot harus menentukan target, arah gerak, arah radar, serta daya tembak yang paling menguntungkan. Oleh karena itu, pendekatan algoritma greedy digunakan karena mampu memilih keputusan lokal terbaik pada setiap turn dengan waktu komputasi yang relatif ringan.

3.1.1 Himpunan Kandidat

Himpunan kandidat merupakan seluruh kemungkinan aksi yang dapat dipilih bot pada suatu kondisi permainan. Pada implementasi bot Robocode, kandidat utama terdiri dari:

1. Musuh yang dapat dijadikan target.
2. Kandidat arah gerak.
3. Kandidat posisi hasil proyeksi movement.
4. Kandidat daya tembak (firepower).
5. Kandidat arah radar.

Setiap bot menggunakan kandidat yang berbeda sesuai strategi heuristic masing-masing.

3.1.2 Himpunan Solusi

Himpunan solusi merupakan keputusan akhir yang dipilih bot setelah seluruh kandidat dievaluasi. Solusi yang dipilih meliputi:

1. Target utama yang akan ditembak.
2. Posisi atau arah gerak terbaik.
3. Besar firepower yang digunakan.
4. Arah radar yang dipakai untuk scanning.

Karena menggunakan pendekatan greedy, solusi dipilih berdasarkan kandidat dengan skor terbaik pada kondisi saat itu.

3.1.3 Himpunan Seleksi

Fungsi seleksi digunakan untuk memilih kandidat terbaik berdasarkan heuristic tertentu. Pada permainan Robocode Tank Royale, fungsi seleksi umumnya menggunakan:

- Jarak musuh,
- Energi musuh,
- Peluang hit,
- Keamanan posisi,
- Ancaman musuh,
- Jarak terhadap dinding,
- Gerakan lateral,
- Peluang kill,
- Serta tingkat bahaya crossfire.

Setiap bot memiliki heuristic berbeda sesuai strategi greedy yang diterapkan.

3.1.4 Fungsi Kelayakan

Fungsi kelayakan digunakan untuk menentukan apakah suatu kandidat layak dipilih. Contoh kondisi kelayakan antara lain:

1. Posisi kandidat masih berada di dalam arena.
2. Posisi tidak terlalu dekat dengan dinding.
3. Firepower tidak melebihi energi bot.
4. Data target masih valid dan belum stale.
5. Kandidat movement tidak menuju area berbahaya.

Jika kandidat tidak memenuhi syarat kelayakan, maka kandidat tersebut akan dieliminasi dari proses greedy.

3.1.4 Fungsi Objektif

Fungsi objektif dari seluruh bot pada tugas besar ini adalah memaksimalkan skor pertandingan Robocode Tank Royale. Skor diperoleh dari:

1. Bullet damage,
2. Bonus kill,
3. Survival score,
4. Last survivor bonus,
5. dan ram damage.

Untuk mencapai objektif tersebut, bot harus mampu menyeimbangkan antara strategi menyerang dan bertahan.

3.2 Proses Mapping Persoalan ke Elemen Greedy

3.2.1 Bot Utama – TankAlbino

Bot utama TankAlbino menggunakan strategi **Hybrid Tactical Greedy** yang menggabungkan target scoring, predictive targeting, adaptive firepower, radar control, duel movement, dan melee movement scoring.

Strategi Greedy

TankAlbino menggunakan beberapa heuristic secara bersamaan untuk memilih keputusan lokal terbaik pada setiap turn permainan.

Heuristic yang Digunakan

1. Heuristic Target Scoring

Target dipilih menggunakan:

- hit chance,
- kill bonus,
- weak bonus,
- threat score,
- ram opportunity,
- melee threat bonus,
- dan age penalty.

2. Heuristic Adaptive Firepower

Firepower disesuaikan berdasarkan:

- jarak target,
- energi bot,
- energi musuh,
- mode melee,
- dan keselarasan gun.

3. Heuristic Predictive Targeting

Bot memprediksi posisi musuh menggunakan:

- arah gerak,
- speed,
- velocity,

- acceleration,
- dan perubahan lateral movement.

4. Heuristic Radar Control

Radar menggunakan:

- radar lock,
- overscan,
- radar sweep melee,
- dan emergency sweep.

5. Heuristic Duel Movement

Pada duel:

- bot melakukan orbit evasion,
- adaptive orbit angle,
- reverse orbit,
- dan jink movement.

6. Heuristic Melee Movement

Pada melee:

- bot mengevaluasi banyak kandidat posisi,
- menghindari crossfire,
- menjaga jarak aman,
- dan menjauh dari centroid ancaman.

7. Heuristic Anti-Wall

Bot selalu memproyeksikan posisi movement agar tidak masuk area berbahaya dekat dinding.

Kelebihan:

1. Sangat adaptif.
2. Seimbang antara offense dan defense.
3. Efektif pada duel maupun melee.
4. Threat handling sangat baik.
5. Movement sulit diprediksi.
6. Radar sangat responsif.

Kekurangan:

1. Kompleksitas sangat tinggi.

2. Banyak parameter heuristic.
3. Membutuhkan tuning intensif.
4. Prediksi masih bisa gagal pada movement ekstrem.

3.2.2 Bot Alternatif 1 – BantengMerah

Bot BantengMerah menggunakan strategi greedy agresif berbasis **ramming dan predictive attack**. Fokus utama bot ini adalah mendekati musuh secepat mungkin dan menghasilkan damage besar melalui kombinasi tembakan jarak dekat dan tabrakan langsung.

Strategi Greedy

Strategi greedy utama BantengMerah adalah memilih aksi yang menghasilkan tekanan ofensif terbesar terhadap musuh. Bot memprioritaskan:

1. mendekati target,
2. mempertahankan radar lock,
3. melakukan zig-zag chase,
4. dan melakukan ramming ketika jarak sudah dekat.

Heuristic yang Digunakan

1. Heuristic Jarak Target

Bot menggunakan heuristic berbasis jarak:

- jika target dekat → mode ramming,
- jika target jauh → mode chase zig-zag.

2. Heuristic Firepower

Firepower dipilih berdasarkan:

- jarak target,
- energi bot,
- mode ramming,
- dan peluang hit.

3. Heuristic Zig-Zag Movement

Movement zig-zag digunakan agar:

- bot tidak mudah diprediksi,
- sekaligus tetap mendekati target.

4. Heuristic Radar Lock

Radar menggunakan:

- full sweep berkala,

- dan radar lock overshoot.

Kelebihan:

1. Sangat agresif.
2. Efektif melawan bot pasif.
3. Damage tinggi pada jarak dekat.
4. Sulit diprediksi karena zig-zag.

Kekurangan:

1. Rentan terhadap bot evasive.
2. Mudah masuk area berbahaya.
3. Kurang efektif pada melee.
4. Boros energi.

3.2.3 Bot Alternatif 2 – AiranScube

Bot Airan menggunakan strategi greedy berbasis **candidate movement scoring** dan **lightning movement**. Fokus utama bot ini adalah memilih posisi terbaik secara cepat menggunakan evaluasi banyak kandidat gerak.

Strategi Greedy

Bot membangkitkan beberapa kandidat movement terhadap posisi musuh, lalu memilih kandidat dengan skor keamanan dan efektivitas tertinggi.

Heuristic yang Digunakan

1. Heuristic Candidate Movement

Bot membuat beberapa kandidat:

- strafing,
- orbit,
- reverse strafe,
- wide evade,
- dan close escape.

2. Heuristic Strafing

Movement lateral mendapat bonus besar karena:

- lebih sulit ditembak,
- efektif melawan linear targeting.

3. Heuristic Preferred Distance

Bot menjaga jarak ideal berdasarkan energi:

- energi tinggi → lebih agresif,
- energi rendah → menjaga jarak lebih jauh.

4. Heuristic Wall Safety

Kandidat dekat dinding diberi penalti besar.

5. Heuristic Center Control

Posisi dekat tengah arena diberi bonus agar movement lebih fleksibel.

6. Heuristic Wiggle Movement

Bot menambahkan osilasi kecil agar movement tidak terlalu linear.

Kelebihan:

1. Movement sangat sulit diprediksi.
2. Survivability tinggi.
3. Efektif melawan predictive targeting.
4. Adaptif terhadap arena.

Kekurangan:

1. Damage output tidak terlalu agresif.
2. Kadang terlalu defensif.
3. Bergantung pada movement scoring.
4. Kurang efektif untuk kill cepat.

3.2.4 Bot Alternatif 3 – Gryffin

Bot Gryffin menggunakan strategi greedy berbasis **tactical mode dan lane evaluation**. Bot ini berfokus pada evaluasi jalur gerak terbaik berdasarkan ancaman arena secara keseluruhan.

Strategi Greedy

Bot membangun banyak kandidat lane movement, kemudian memilih lane dengan skor keselamatan tertinggi berdasarkan:

- posisi musuh,
- ancaman,
- wall safety,
- pressure,
- dan tactical mode aktif.

Heuristic yang Digunakan

1. Heuristic Tactical Mode

Bot memiliki beberapa mode:

- Stabilize,
- Hunt,
- BreakLine,
- Pressure,
- Recover.

2. Heuristic Threat Index

Setiap musuh memiliki:

- threat index,
- pressure index,
- dan finish index.

3. Heuristic Lane Scoring

Setiap lane dievaluasi berdasarkan:

- wall safety,
- jarak ideal,
- centroid escape,
- lateral movement,
- revisit penalty,
- pressure,
- dan close danger.

4. Heuristic Revisit Penalty

Bot menghindari bergerak ke area yang sama secara berulang.

5. Heuristic Boundary Stabilization

Jika terlalu dekat dinding, bot masuk mode stabilize dan bergerak ke tengah arena.

Kelebihan:

1. Sangat stabil pada melee.
2. Threat analysis sangat baik.
3. Movement adaptif terhadap arena.
4. Menghindari pola repetitif.

Kekurangan:

1. Kompleksitas heuristic tinggi.
2. Movement kadang terlalu konservatif.
3. Membutuhkan evaluasi kandidat yang banyak.
4. Respons agresif lebih lambat.

3.3 Analisis Efisiensi dan Efektivitas

Keempat bot menggunakan pendekatan greedy, tetapi masing-masing memiliki fokus heuristic berbeda.

Bot	Fokus Utama	Kelebihan	Kekurangan
TankAlbino	Hybrid tactical greedy	Paling seimbang	Kompleksitas tinggi
BantengMerah	Aggressive ramming	Damage tinggi	Rentan defensif
AiranScube	Lightning movement	Survivability tinggi	Kurang agresif
Gryffin	Tactical lane scoring	Stabil dan aman	Kompleks dan lambat

Analisis TankAlbino

TankAlbino memiliki performa paling stabil karena mampu menggabungkan offensive heuristic dan defensive heuristic secara seimbang. Bot mampu memilih target efektif sekaligus mempertahankan survivability pada duel maupun melee.

Analisis BantengMerah

BantengMerah efektif dalam duel agresif karena mampu menghasilkan damage besar pada jarak dekat. Namun, strategi greedy ini kurang optimal pada kondisi melee karena bot terlalu fokus mengejar target.

Analisis AiranScube

Airan memiliki movement terbaik di antara seluruh bot karena movement candidate scoring membuat gerakan sulit diprediksi. Namun, strategi ini cenderung defensif sehingga damage output lebih rendah.

Analisis Gryffin

Gryffin sangat baik dalam pengendalian posisi arena dan penghindaran ancaman. Namun, kompleksitas lane scoring membuat respons ofensif terkadang lebih lambat.

3.4 Strategi Greedy yang Dipilih

Strategi greedy yang dipilih sebagai solusi utama adalah strategi **Hybrid Tactical Greedy** pada bot TankAlbino.

Alasan Pemilihan

1. Memiliki keseimbangan terbaik antara offense dan defense.
2. Efektif pada duel maupun melee.

3. Menggunakan target scoring paling lengkap.
4. Memiliki movement adaptif dan anti-crossfire.
5. Menggunakan predictive targeting yang lebih akurat.
6. Memiliki radar control yang stabil.
7. Mampu menangani ancaman dari banyak arah.

Justifikasi

Dibandingkan bot alternatif lainnya:

1. BantengMerah terlalu agresif,
2. Airan terlalu defensif,
3. Gryffin terlalu kompleks pada movement,
4. sedangkan TankAlbino mampu menggabungkan kelebihan ketiganya.

Strategi Hybrid Tactical Greedy pada TankAlbino dinilai paling optimal untuk Robocode Tank Royale karena mampu mengambil keputusan lokal terbaik secara adaptif berdasarkan kondisi arena, ancaman, dan peluang serangan pada setiap turn permainan.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Algoritma Greedy

4.1.1 Pseudocode Main-bot

Bot utama yang diimplementasikan bernama **TankAlbino**. Bot ini menggunakan strategi **Hybrid Tactical Greedy**, yaitu strategi greedy yang mengambil keputusan terbaik pada setiap turn berdasarkan skor keuntungan tertinggi. Keputusan yang diambil meliputi pemilihan target, arah radar, daya tembak, titik bidik, dan arah pergerakan.

Strategi greedy pada bot ini berorientasi pada dua tujuan utama, yaitu menghasilkan damage sebesar mungkin dan mempertahankan bot agar tetap hidup selama mungkin. Hal ini sesuai dengan objektif permainan Robocode Tank Royale, yaitu memperoleh skor akhir tertinggi melalui bullet damage, survival score, bonus kill, last survivor bonus, dan ram damage.

```
// =====  
// INISIALISASI  
// =====  
  
Inisialisasi:  
enemies ← dictionary kosong  
orbitSign ← 1  
radarSign ← 1  
lastTargetId ← -1  
duelJinkUntil ← 0  
forceCenterUntil ← 0  
lastBulletHitTurn ← -999  
  
// =====  
// LOOP UTAMA BOT  
// =====  
  
SELAMA bot berjalan:  
target ← SelectGreedyTarget()  
  
IF target tidak ditemukan THEN  
SearchForEnemy()  
ELSE  
ControlRadar(target)  
ControlGunAndFire(target)  
ControlMovement(target)  
ENDIF  
  
Go()
```



```

// =====
// PEMILIHAN TARGET GREEDY
// =====

FUNGSI SelectGreedyTarget():
daftarMusuh ← GetFreshEnemies(MaxTargetAge)

bestTarget ← null
bestScore ←  $-\infty$ 

UNTUK setiap enemy dalam daftarMusuh:
score ← GreedyTargetScore(enemy)

IF enemy.id = lastTargetId THEN
score ← score + bonus_kontinuitas
ENDIF

IF score > bestScore THEN
bestScore ← score
bestTarget ← enemy
ENDIF
ENDFOR

IF bestTarget tidak null THEN
lastTargetId ← bestTarget.id
ENDIF

RETURN bestTarget

// =====
// PERHITUNGAN SKOR TARGET
// =====

FUNGSI GreedyTargetScore(enemy):

distance ← jarak bot ke enemy
gunBearing ← sudut gun terhadap enemy

power ← EstimateFirepower(enemy, distance, gunBearing)
damage ← BulletDamage(power)

hitByDistance ←  $1 / (1 + \text{distance} / 360)$ 

```

```

hitByAlignment ←
1 - min(gunBearing, 90) / 90

hitChance ←
0.65 × hitByDistance
+ 0.35 × hitByAlignment

IF enemy.energy <= damage THEN
killBonus ← 5.5
ELSE
killBonus ← 0
ENDIF

weakBonus ←
(100 - enemy.energy) / 45

closeThreat ←
faktor energi dan jarak musuh

ramOpportunity ←
bonus jika kondisi ram memungkinkan

meleeThreatBonus ←
bonus jika musuh dekat
dari samping atau belakang

agePenalty ←
penalti data lama

RETURN
hitChance × 5
+ killBonus
+ weakBonus
+ closeThreat × 1.4
+ ramOpportunity
+ meleeThreatBonus
- agePenalty

// =====
// KONTROL RADAR
// =====

```

FUNGSI ControlRadar(target):

IF mode melee DAN turn tertentu THEN

RadarTurnRate $\leftarrow$ putaran maksimum

RETURN

ENDIF

bearing $\leftarrow$ RadarBearingTo(target)

turn $\leftarrow$ bearing $\times$ 2.2

IF |turn| < 4 THEN

turn $\leftarrow$ 4 $\times$ radarSign

ENDIF

turn $\leftarrow$

batasi sesuai MaxRadarTurnRate

radarSign $\leftarrow$ arah turn

RadarTurnRate $\leftarrow$ turn

// =====

// KONTROL GUN DAN PENEMBAKAN

// =====

FUNGSI ControlGunAndFire(target):

distance $\leftarrow$ jarak ke target

bearing $\leftarrow$ sudut gun terhadap target

power $\leftarrow$

EstimateFirepower(target, distance, bearing)

IF power < 0.1 THEN

RETURN

ENDIF

aimPoint $\leftarrow$

SelectAimPoint(target, power)

gunBearing $\leftarrow$

GunBearingTo(aimPoint)

```

GunTurnRate ←
batasi gunBearing

fireWindow ←
FireWindow(jarak ke aimPoint)

IF GunHeat = 0
DAN |gunBearing| <= fireWindow
DAN Energy > power + 0.25
THEN
SetFire(power)
ENDIF

// =====
// ESTIMASI FIREPOWER
// =====

FUNGSI EstimateFirepower(target, distance, gunBearing):

maxAffordable ←
minimum(3, Energy - 0.2)

IF maxAffordable < 0.1 THEN
RETURN 0
ENDIF

IF distance < 140 THEN
power ← 3.0
ELSE IF distance < 300 THEN
power ← 2.25
ELSE IF distance < 550 THEN
power ← 1.55
ELSE IF distance < 800 THEN
power ← 1.0
ELSE
power ← 0.65
ENDIF

IF target.energy rendah THEN
power ← secukupnya untuk kill
ENDIF

IF gun tidak sejajar ATAU target terlalu jauh THEN

```


kurangi power

ENDIF

IF target zig-zag THEN

gunakan power kecil-menengah

ENDIF

IF energi bot rendah THEN

batasi power

ENDIF

IF mode melee THEN

hemat energi

ENDIF

RETURN power

// =====

// PEMILIHAN TITIK BIDIK

// =====

FUNGSI SelectAimPoint(target, firepower):

IF mode melee

ATAU target tidak zig-zag

THEN

RETURN PredictEnemyPosition(target, firepower)

ENDIF

IF target baru berbalik arah THEN

gunakan lead pendek

IF head-on lebih baik THEN

RETURN posisi target saat ini

ENDIF

ELSE

gunakan observed prediction

ENDIF

RETURN titik prediksi terbaik

// =====

// PREDIKSI POSISI MUSUH

// =====

FUNGSI PredictEnemyPosition(target, firepower):

bulletSpeed $\leftarrow$ kecepatan peluru

px $\leftarrow$ target.x

py $\leftarrow$ target.y

ULANGI 4 kali:

travelTime $\leftarrow$

jarak(px, py) / bulletSpeed

px $\leftarrow$

target.x

+ cos(target.direction)

$\times$ target.speed

$\times$ travelTime

py $\leftarrow$

target.y

+ sin(target.direction)

$\times$ target.speed

$\times$ travelTime

batasi px dan py agar tetap dalam arena

ENDULANGI

RETURN AimPoint(px, py)

// =====

// PREDIKSI OBSERVED MOVEMENT

// =====

FUNGSI PredictObservedEnemyPosition(

target,

firepower,

leadScale

):

bulletSpeed $\leftarrow$ kecepatan peluru

vx $\leftarrow$ velocityX target

vy $\leftarrow$ velocityY target

```
IF velocity terlalu kecil THEN
hitung velocity dari arah dan speed
ENDIF
```

```
px ← target.x
py ← target.y
```

```
ULANGI 3 kali:
travelTime ←
jarak(px, py) / bulletSpeed
```

```
px ←
target.x
+ vx × travelTime × leadScale
```

```
py ←
target.y
+ vy × travelTime × leadScale
```

```
batasi posisi dalam arena
ENDULANGI
```

```
RETURN AimPoint(px, py)
```

```
// =====
// KONTROL MOVEMENT
// =====
```

```
FUNGSI ControlMovement(target):
```

```
IF mode melee THEN
ControlMeleeMovement(target)
RETURN
ENDIF
```

```
distance ← jarak ke target
```

```
nearWall ←
bot dekat dinding
```

```
jinkMode ←
bot baru terkena peluru
```

ATAU musuh baru menembak

```
IF terlalu lama tidak reverse THEN  
ReverseOrbit()  
ENDIF
```

```
IF nearWall THEN  
desiredHeading ← arah ke tengah arena
```

```
ELSE IF ShouldRam(target, distance) THEN  
desiredHeading ← arah langsung ke target
```

```
ELSE
```

```
toEnemy ← arah ke target
```

```
IF distance < 170 THEN  
orbitAngle ← 138
```

```
ELSE IF distance > 560 THEN  
orbitAngle ← 42
```

```
ELSE  
orbitAngle ← 92  
ENDIF
```

```
IF jinkMode THEN  
ubah orbitAngle  
ENDIF
```

```
desiredHeading ←  
toEnemy + orbitAngle × orbitSign
```

```
IF proyeksi keluar zona aman THEN  
desiredHeading ← arah ke tengah  
ENDIF  
ENDIF
```

```
tentukan speedFactor
```

```
MoveToward(desiredHeading, speedFactor)
```

```

// =====
// MOVEMENT MODE MELEE
// =====

FUNGSI ControlMeleeMovement(target):

    threats ← daftar musuh aktif

    IF target bisa di-ram THEN
        bergerak langsung ke target
    RETURN
    ENDIF

    bestHeading ← Direction
    bestScore ←  $-\infty$ 

    UNTUK 16 arah kandidat:

        projected ←
        posisi hasil proyeksi heading

        score ←
        ScoreMeleePosition(projected)

        IF score > bestScore THEN
            bestScore ← score
            bestHeading ← heading
        ENDIF

    ENDFOR

    tambahkan kandidat:
    arah ke tengah arena
    arah keluar crossfire
    arah menjauh dari centroid musuh

    pilih heading dengan score tertinggi

    MoveToward(bestHeading, speedFactor)

// =====
// SKOR POSISI MELEE
// =====

```

FUNGSI ScoreMeleePosition(point):

IF point keluar arena THEN

RETURN $-\infty$

ENDIF

wallScore $\leftarrow$

skor jarak dari dinding

nearestScore $\leftarrow$

skor jarak dari musuh terdekat

averageScore $\leftarrow$

skor rata-rata jarak semua musuh

lateralScore $\leftarrow$

skor gerak menyamping

targetDistanceScore $\leftarrow$

skor jarak ideal target

centerScore $\leftarrow$

skor kedekatan tengah arena

closePenalty $\leftarrow$

penalti terlalu dekat musuh

crossfirePenalty $\leftarrow$

penalti garis tembak silang

RETURN

wallScore

+ nearestScore $\times 2.4$

+ averageScore $\times 1.25$

+ lateralScore $\times 1.35$

+ targetDistanceScore $\times 0.85$

+ centerScore $\times 0.45$

- closePenalty $\times 2.7$

- crossfirePenalty $\times 3.0$

// =====

// GERAK MENUJU HEADING

// =====

FUNGSI MoveToward(moveHeading, speedFactor):

turn $\leftarrow$

selisih heading terhadap body

speed $\leftarrow$

MaxSpeed $\times$ speedFactor

IF |turn| > 100 THEN

IF reverse tidak aman THEN

speed $\leftarrow$ speed kecil

ELSE

turn $\leftarrow$ turn + 180

speed $\leftarrow$ -speed

ENDIF

ENDIF

TurnRate $\leftarrow$ batasi turn

TargetSpeed $\leftarrow$ speed

// =====

// CEK REVERSE AMAN

// =====

FUNGSI IsReverseUnsafe():

projected $\leftarrow$

posisi hasil reverse

IF projected dekat dinding THEN

RETURN true

ENDIF

UNTUK setiap musuh:

IF reverse mendekati musuh samping/belakang THEN

RETURN true

ENDIF

ENDFOR

```
RETURN false
```

```
// =====  
// SKOR BAHAYA CROSSFIRE  
// =====
```

```
FUNGSI ScoreCrossfireDanger(point):
```

```
danger ← 0
```

```
UNTUK setiap pasangan musuh:
```

```
hitung jarak point ke garis antar musuh
```

```
IF point berada di koridor crossfire THEN
```

```
tambah danger
```

```
ENDIF
```

```
ENDFOR
```

```
RETURN danger
```

```
// =====  
// ESCAPE DARI CROSSFIRE  
// =====
```

```
FUNGSI EscapeHeadingFromCrossfire():
```

```
cari koridor crossfire paling berbahaya
```

```
RETURN arah menjauh dari koridor
```

```
// =====  
// ESCAPE DARI KERUMUNAN MUSUH  
// =====
```

```
FUNGSI EscapeHeadingFromThreatCentroid():
```

```
hitung centroid posisi musuh
```

```
RETURN arah berlawanan dari centroid
```

```
// =====  
// DETEKSI RAM  
// =====
```



```
FUNGSI ShouldRam(target, distance):
```

```
RETURN
```

```
musuh tinggal sedikit
```

```
DAN energi kita jauh lebih besar
```

```
DAN energi target rendah
```

```
DAN target dekat
```

```
// =====
```

```
// MODE MELEE
```

```
// =====
```

```
FUNGSI IsMeleeMode():
```

```
RETURN EnemyCount > 2
```

```
// =====
```

```
// REVERSE ORBIT
```

```
// =====
```

```
FUNGSI ReverseOrbit():
```

```
IF terlalu cepat reverse THEN
```

```
RETURN
```

```
ENDIF
```

```
orbitSign ← -orbitSign
```

```
lastReverseTurn ← TurnNumber
```

```
// =====
```

```
// RESET STATE
```

```
// =====
```

```
FUNGSI ResetRoundState():
```

```
enemies.clear()
```

```
orbitSign ← 1
```

```
radarSign ← 1
```

```
lastReverseTurn ← -999
```

```
forceCenterUntil ← 0
```

```
lastTargetId ← -1  
duelJinkUntil ← 0  
lastBulletHitTurn ← -9
```

4.1.2 Pseudocode Alternative Bot 1

Bot alternatif pertama yang diimplementasikan bernama **BantengMerah**. Bot ini menggunakan strategi **Greedy Closest Target Rammer**, yaitu strategi greedy yang memilih musuh terdekat sebagai target utama pada setiap turn. Musuh yang memiliki jarak paling kecil dianggap sebagai kandidat terbaik karena paling cepat dijangkau, paling mudah ditekan, dan paling memungkinkan untuk menghasilkan ram damage.

Strategi greedy pada bot ini berorientasi pada gaya bermain agresif. Keputusan utama yang diambil meliputi pemilihan target terdekat, penguncian radar ke target, pemilihan daya tembak berdasarkan jarak, prediksi posisi musuh, dan movement mengejar musuh dengan pola zig-zag atau ramming. Strategi ini bertujuan memperoleh skor dari bullet damage, ram damage, ram damage bonus, dan peluang membunuh musuh dalam jarak dekat.

```
// =====  
// INISIALISASI  
// =====  
  
Inisialisasi:  
enemies ← dictionary kosong  
target ← null  
radarSweepDir ← 1  
lastSweepTurn ← 0  
sweeping ← true  
sweepAccumulated ← 0  
lastRadarDir ← RadarDirection  
zigzagDir ← 1  
zigzagCounter ← 0  
zigzagPhaseLimit ← ZigzagPhaseTurns  
isRamming ← false  
ramSustainCounter ← 0  
  
// =====  
// LOOP UTAMA BOT  
// =====  
  
SELAMA bot berjalan:  
ExecuteRadarControl()  
  
IF HasFreshTarget() THEN
```

```

ExecuteRamFire(target)
ExecuteRamChase(target)
ELSE
target ← null
isRamming ← false
ramSustainCounter ← 0
ExecuteSearchPattern()
ENDIF

```

```

Go()
ENDSELAMA

```

```

// =====
// KONTROL RADAR
// =====

```

```

FUNGSI ExecuteRadarControl():

```

```

radarDelta ← perubahan arah radar dari turn sebelumnya
sweepAccumulated ← sweepAccumulated + radarDelta
lastRadarDir ← RadarDirection

```

```

IF TurnNumber - lastSweepTurn >= SweepInterval THEN
sweeping ← true
sweepAccumulated ← 0
lastSweepTurn ← TurnNumber
ENDIF

```

```

IF sweeping THEN
RadarTurnRate ←  $45 \times \text{radarSweepDir}$ 

```

```

IF sweepAccumulated >= 360 THEN
sweeping ← false
sweepAccumulated ← 0
radarSweepDir ← -radarSweepDir
ENDIF

```

```

ELSE IF target tidak null THEN
radarBearing ← RadarBearingTo(target)

```

```

overshoot ← arah radarBearing  $\times$  22

```

```

IF target.speed > 4 THEN
overshoot  $\leftarrow$  overshoot  $\times$  1.5
ENDIF

RadarTurnRate  $\leftarrow$  radarBearing + overshoot

ELSE
sweeping  $\leftarrow$  true
sweepAccumulated  $\leftarrow$  0
ENDIF

// =====
// MOVEMENT GREEDY RAM CHASE
// =====

FUNGSI ExecuteRamChase(target):

IF IsNearWall() THEN
isRamming  $\leftarrow$  false
ramSustainCounter  $\leftarrow$  0

desiredHeading  $\leftarrow$  arah ke tengah arena
TurnRate  $\leftarrow$  desiredHeading
MoveForward(200)
TargetSpeed  $\leftarrow$  8
zigzagCounter  $\leftarrow$  0
RETURN
ENDIF

distance  $\leftarrow$  jarak bot ke target
bearingToTarget  $\leftarrow$  BearingTo(target)

IF ramSustainCounter > 0 THEN
ramSustainCounter  $\leftarrow$  ramSustainCounter - 1
ENDIF

shouldRam  $\leftarrow$ 
distance  $\leq$  RamEngageRange
ATAU ramSustainCounter > 0

```

```

IF shouldRam THEN
isRamming ← true

TurnRate ← bearingToTarget
MoveForward(distance + 300)
TargetSpeed ← 8
zigzagCounter ← 0

ELSE
isRamming ← false
zigzagCounter ← zigzagCounter + 1

IF zigzagCounter >= zigzagPhaseLimit THEN
zigzagCounter ← 0
zigzagDir ← -zigzagDir
zigzagPhaseLimit ← nilai acak antara 8 sampai 16
ENDIF

zigzagBearing ←
bearingToTarget + ZigzagAngle × zigzagDir

TurnRate ← zigzagBearing
MoveForward(distance)
TargetSpeed ← 8
ENDIF

```

```

// =====
// KONTROL GUN DAN PENEMBAKAN
// =====

```

```

FUNGSI ExecuteRamFire(target):

```

```

IF GunHeat ≠ 0 THEN
RETURN
ENDIF

```

```

IF Energy < 1.0 THEN
RETURN
ENDIF

```

```

distance ← jarak bot ke target

```

```
IF Energy < EnergyCritical THEN
estimatedFirepower ← 1.0
ELSE IF isRamming THEN
estimatedFirepower ← 3.0
ELSE IF distance < MediumRange THEN
estimatedFirepower ← 2.0
ELSE
estimatedFirepower ← 1.5
ENDIF
```

```
predictedPoint ← PredictTargetPosition(target, estimatedFirepower)
```

```
gunBearing ← GunBearingTo(predictedPoint)
GunTurnRate ← gunBearing
```

```
IF isRamming THEN
aimThreshold ← 30
ELSE IF distance < 200 THEN
aimThreshold ← 15
ELSE
aimThreshold ← 8
ENDIF
```

```
IF |gunBearing| > aimThreshold THEN
RETURN
ENDIF
```

```
fireOptions ← GetFirepowerOptions(distance, isRamming, Energy)
```

```
bestFirepower ← -1
bestScore ←  $-\infty$ 
```

UNTUK setiap firepower dalam fireOptions:

```
IF Energy <= firepower + 0.2 THEN
lanjut ke firepower berikutnya
ENDIF
```

```
hitChance ← EstimateHitChance(distance, |gunBearing|, target.speed, firepower)
```

```

IF isRamming THEN
minimumHitChance ← 0.05
ELSE
minimumHitChance ← 0.10
ENDIF

IF hitChance < minimumHitChance THEN
lanjut ke firepower berikutnya
ENDIF

damage ← BulletDamage(firepower)
energyPenalty ← firepower × faktor_kondisi_energi

score ← damage × hitChance - energyPenalty

IF score > bestScore THEN
bestScore ← score
bestFirepower ← firepower
ENDIF

ENDFOR

IF bestFirepower > 0 THEN
finalAimPoint ← PredictTargetPosition(target, bestFirepower)
GunTurnRate ← GunBearingTo(finalAimPoint)
SetFire(bestFirepower)

ELSE IF |gunBearing| < 20 DAN Energy > 1.2 THEN
SetFire(minimum(1.0, Energy - 0.2))
ENDIF

// =====
// PEMILIHAN OPSI FIREPOWER
// =====

FUNGSI GetFirepowerOptions(distance, isRamming, energy):

IF energy < EnergyCritical THEN
RETURN {1.0}

ELSE IF isRamming ATAU distance < CloseRange THEN

```



```
RETURN {2.5, 3.0}
```

```
ELSE IF distance < MediumRange THEN
```

```
RETURN {1.5, 2.0, 2.5}
```

```
ELSE
```

```
RETURN {1.0, 1.5}
```

```
ENDIF
```

```
// =====
```

```
// PREDIKSI POSISI TARGET
```

```
// =====
```

```
FUNGSI PredictTargetPosition(target, firepower):
```

```
predX ← target.x
```

```
predY ← target.y
```

```
ULANGI 5 kali:
```

```
bulletSpeed ← 20 - 3 × firepower
```

```
travelTime ← jarak bot ke prediksi / bulletSpeed
```

```
predX ← target.x + sin(target.direction) × target.speed × travelTime
```

```
predY ← target.y + cos(target.direction) × target.speed × travelTime
```

```
batasi predX dan predY agar tetap dalam arena aman
```

```
ENDULANGI
```

```
RETURN Point(predX, predY)
```

```
// =====
```

```
// ESTIMASI PELUANG HIT
```

```
// =====
```

```
FUNGSI EstimateHitChance(distance, gunOffset, enemySpeed, firepower):
```

```
bulletSpeed ← 20 - 3 × firepower
```

```
distanceFactor ←
```

```
batasi(1.2 - distance / (bulletSpeed × 35), 0.1, 1.0)
```

```

aimFactor ←
batasi(1 - gunOffset / 35, 0.05, 1.0)

speedFactor ←
batasi(1 - |enemySpeed| / 10, 0.2, 1.0)

RETURN
batasi(distanceFactor × aimFactor × speedFactor, 0.03, 0.97)

// =====
// SEARCH PATTERN
// =====

FUNGSI ExecuteSearchPattern():

IF IsNearWall() THEN
desiredHeading ← arah ke tengah arena
MoveForward(200)
ELSE
desiredHeading ← 5 derajat
MoveForward(100)
ENDIF

TurnRate ← desiredHeading
TargetSpeed ← 6

// =====
// EVENT: MUSUH TERDETEKSI
// =====

FUNGSI OnScannedBot(event):

distance ← jarak bot ke musuh

enemies[event.id] ← EnemyInfo(
id,
x,
y,
energy,
direction,
speed,

```

```

distance,
turnNumber
)

SelectClosestTarget()

// =====
// PEMILIHAN TARGET TERDEKAT
// =====

FUNGSI SelectClosestTarget():

closest ← null
minDistance ← ∞

UNTUK setiap enemy dalam enemies:

IF enemy.energy ≤ 0 THEN
    lanjut ke enemy berikutnya
ENDIF

IF TurnNumber - enemy.lastSeenTurn > StaleThreshold THEN
    lanjut ke enemy berikutnya
ENDIF

IF enemy.distance < minDistance THEN
    minDistance ← enemy.distance
    closest ← enemy
ENDIF

ENDFOR

target ← closest

// =====
// EVENT: MENABRAK BOT
// =====

FUNGSI OnHitBot(event):

IF GunHeat = 0 DAN Energy > MaxFirepower + 0.1 THEN

```

```
SetFire(MaxFirepower)
```

```
ENDIF
```

```
ramSustainCounter ← RamSustainTurns
```

```
isRamming ← true
```

```
zigzagCounter ← 0
```

```
zigzagPhaseLimit ← ZigzagPhaseTurns
```

```
MoveForward(250)
```

```
TargetSpeed ← 8
```

```
Go()
```

```
// =====
```

```
// EVENT: TERKENA PELURU
```

```
// =====
```

```
FUNGSI OnHitByBullet(event):
```

```
IF bukan sedang ramming THEN
```

```
zigzagDir ← -zigzagDir
```

```
zigzagCounter ← 0
```

```
zigzagPhaseLimit ← nilai acak antara 6 sampai 12
```

```
ENDIF
```

```
IF Energy < EnergyCritical DAN bukan sedang ramming THEN
```

```
MoveBack(80)
```

```
TargetSpeed ← -4
```

```
Go()
```

```
ENDIF
```

```
// =====
```

```
// EVENT: MENABRAK DINDING
```

```
// =====
```

```
FUNGSI OnHitWall(event):
```

```
isRamming ← false
```

```
ramSustainCounter ← 0
```

```
MoveBack(80)
```

```
TurnRate ← arah ke tengah arena
```

```

sweeping ← true
sweepAccumulated ← 0
zigzagDir ← -zigzagDir
zigzagCounter ← 0
Go()

// =====
// EVENT: BOT MUSUH MATI
// =====

FUNGSI OnBotDeath(event):

hapus enemy dari dictionary enemies

IF target.id = enemy yang mati THEN
target ← null
isRamming ← false
ramSustainCounter ← 0
sweeping ← true
sweepAccumulated ← 0
ENDIF

// =====
// CEK TARGET VALID
// =====

FUNGSI HasFreshTarget():

RETURN
target tidak null
DAN TurnNumber - target.lastSeenTurn <= StaleThreshold

// =====
// CEK DEKAT DINDING
// =====

FUNGSI IsNearWall():

RETURN
X < WallMargin

```

```
ATAU  $Y < \text{WallMargin}$   
ATAU  $X > \text{ArenaWidth} - \text{WallMargin}$   
ATAU  $Y > \text{ArenaHeight} - \text{WallMargin}$ 
```

4.1.3 Pseudocode Alternative Bot 2

Bot alternatif kedua yang diimplementasikan bernama **AiranScube**. Bot ini menggunakan strategi **Greedy Evasive Strafing**, yaitu strategi greedy yang memilih kandidat gerakan terbaik berdasarkan skor posisi. Berbeda dari BantengMerah yang agresif mengejar musuh, AiranScube lebih berfokus pada menjaga jarak, bergerak menyamping, dan menghindari tembakan lawan.

Strategi greedy pada bot ini berorientasi pada survival dan konsistensi damage. Bot membentuk beberapa kandidat posisi, menghitung skor setiap kandidat berdasarkan jarak ideal dari musuh, keamanan dari dinding, kualitas strafing, kedekatan terhadap tengah arena, dan bahaya jarak dekat. Kandidat dengan skor tertinggi dipilih sebagai arah gerak pada turn tersebut.

```
// =====  
// INISIALISASI  
// =====  
  
Inisialisasi:  
target ← null  
moveFlip ← 1  
lastDirectionSwitchTurn ← -999  
radarSweepDirection ← 1  
wasTrackingTarget ← false  
velocityTurnCounter ← 0  
velocityMode ← 1  
fullScanMode ← false  
fullScanStartTurn ← -999  
lastObservedTurn ← -1  
wiggle ← 0  
wiggleCounter ← 0  
rng ← Random()
```

```
// =====  
// LOOP UTAMA BOT  
// =====  
  
SELAMA bot berjalan:  
  
ResetRoundStateIfNeeded()  
  
hasTarget ← HasFreshTarget()  
  
IF hasTarget THEN  
  
    targetAge ← TurnNumber - target.lastSeenTurn  
  
    ExecuteLightningMovement(target)  
    ExecuteGreedyAimAndFire(target)  
  
    IF fullScanMode THEN  
  
        IF TurnNumber - fullScanStartTurn >= FullScanTimeout THEN  
            fullScanMode ← false  
        ELSE  
            ExecuteWideRadarSweep()  
        ENDIF  
  
        ELSE IF targetAge <= RadarUnlockTurns THEN  
            LockRadar(target)  
  
        ELSE  
            ExecuteWideRadarSweep()  
        ENDIF  
  
        wasTrackingTarget ← true  
  
    ELSE  
  
        IF wasTrackingTarget THEN  
            target ← null  
        ENDIF  
  
        ExecuteSearchPattern()
```

```
wasTrackingTarget ← false
```

```
ENDIF
```

```
Go()
```

```
ENDSELAMA
```

```
// =====
```

```
// FULL SCAN
```

```
// =====
```

```
FUNGSI BeginFullScan():
```

```
fullScanMode ← true
```

```
fullScanStartTurn ← TurnNumber
```

```
RadarTurnRate ←  $360 \times \text{radarSweepDirection}$ 
```

```
// =====
```

```
// RESET STATE RONDE
```

```
// =====
```

```
FUNGSI ResetRoundStateIfNeeded():
```

```
IF lastObservedTurn  $\geq$  0
```

```
DAN TurnNumber < lastObservedTurn
```

```
THEN
```

```
target ← null
```

```
wasTrackingTarget ← false
```

```
fullScanMode ← true
```

```
fullScanStartTurn ← TurnNumber
```

```
velocityMode ← 1
```

```
velocityTurnCounter ← 0
```

```
ENDIF
```

```
lastObservedTurn ← TurnNumber
```

```
// =====
```

```
// MOVEMENT GREEDY LIGHTNING
```

```
// =====
```

```
FUNGSI ExecuteLightningMovement(enemy):
```


velocityTurnCounter $\leftarrow$ velocityTurnCounter + 1

wiggleCounter $\leftarrow$ wiggleCounter + 1

IF velocityTurnCounter mod 32 = 0

ATAU enemy terlalu dekat DAN peluang acak terpenuhi

THEN

velocityMode $\leftarrow$ -velocityMode

velocityTurnCounter $\leftarrow$ 0

ENDIF

IF terlalu lama tidak mengganti arah strafing

DAN peluang acak terpenuhi

THEN

moveFlip $\leftarrow$ -moveFlip

lastDirectionSwitchTurn $\leftarrow$ TurnNumber

ENDIF

toEnemy $\leftarrow$ DirectionTo(enemy)

wiggle $\leftarrow$ sin(wiggleCounter $\times$ 0.85) $\times$ 18

candidates $\leftarrow$ {

BuildCandidate(toEnemy + 85 $\times$ moveFlip + wiggle, 165),

BuildCandidate(toEnemy + 110 $\times$ moveFlip + wiggle, 150),

BuildCandidate(toEnemy + 65 $\times$ moveFlip + wiggle, 140),

BuildCandidate(toEnemy + 135 $\times$ moveFlip, 130),

BuildCandidate(toEnemy - 85 $\times$ moveFlip + wiggle, 155),

BuildCandidate(toEnemy + 155 $\times$ moveFlip, 120),

BuildCandidate(toEnemy + 175, 105),

BuildCandidate(toEnemy + 45 $\times$ moveFlip, 115),

BuildCandidate(toEnemy - 45 $\times$ moveFlip, 110)

}

bestCandidate $\leftarrow$ candidates[0]

bestScore $\leftarrow$ $-\infty$

UNTUK setiap candidate dalam candidates:

score $\leftarrow$ ScoreLightningCandidate(candidate, enemy)

```

angleDiff ←
|selisih candidate.heading terhadap arah musuh|

IF angleDiff > 65 DAN angleDiff < 115 THEN
score ← score + bonus_straifing
ENDIF

IF score > bestScore THEN
bestScore ← score
bestCandidate ← candidate
ENDIF

ENDFOR

IF velocityMode = 1 THEN
desiredVelocity ← 8
ELSE IF velocityMode = -1 THEN
desiredVelocity ← -8
ELSE
desiredVelocity ← 6.5
ENDIF

IF turn tertentu DAN peluang acak terpenuhi THEN
desiredVelocity ← desiredVelocity × 0.32
ENDIF

IF sudut gerak terhadap musuh menghasilkan strafing baik THEN
strafeFactor ← 1.65
ELSE
strafeFactor ← 1.0
ENDIF

TurnRate ← 4.8 × moveFlip × strafeFactor

DriveToVelocity(
bestCandidate.targetX,
bestCandidate.targetY,
desiredVelocity
)

// =====

```

```
// SKOR KANDIDAT MOVEMENT
```

```
// =====
```

```
FUNGSI ScoreLightningCandidate(candidate, enemy):
```

```
distanceToEnemy  $\leftarrow$  jarak candidate ke enemy
```

```
IF Energy > 45 THEN
```

```
preferredDistance  $\leftarrow$  275
```

```
ELSE IF Energy > 25 THEN
```

```
preferredDistance  $\leftarrow$  340
```

```
ELSE
```

```
preferredDistance  $\leftarrow$  420
```

```
ENDIF
```

```
distanceScore  $\leftarrow$ 
```

```
 $265 - |distanceToEnemy - preferredDistance| \times 0.68$ 
```

```
wallScore  $\leftarrow$  WallSafety(candidate.x, candidate.y)  $\times$  2.25
```

```
strafeAngle  $\leftarrow$ 
```

```
sudut gerak candidate terhadap enemy
```

```
strafeScore  $\leftarrow$ 
```

```
 $(1 - |strafeAngle - 90| / 90) \times 145$ 
```

```
centerScore  $\leftarrow$ 
```

```
 $48 - \text{jarak candidate ke tengah arena} \times 0.09$ 
```

```
IF distanceToEnemy < 115 THEN
```

```
closeDanger  $\leftarrow$  -230
```

```
ELSE
```

```
closeDanger  $\leftarrow$  0
```

```
ENDIF
```

```
randomScore  $\leftarrow$  nilai acak kecil
```

```
RETURN
```

```
distanceScore
```

```
+ wallScore
```

```
+ strafeScore
```

```

+ centerScore
+ closeDanger
+ randomScore

// =====
// GERAK MENUJU TARGET VELOCITY
// =====

FUNGSI DriveToVelocity(targetX, targetY, velocity):

bearing ← BearingTo(targetX, targetY)
distance ← DistanceTo(targetX, targetY)

IF |bearing| <= 90 THEN
TurnRate ← bearing

IF velocity >= 0 THEN
MoveForward(distance)
ELSE
MoveBack(distance)
ENDIF

ELSE
backBearing ← bearing + 180 yang dinormalisasi
TurnRate ← backBearing

IF velocity >= 0 THEN
MoveBack(distance)
ELSE
MoveForward(distance)
ENDIF

ENDIF

TargetSpeed ← velocity

// =====
// MEMBANGUN KANDIDAT GERAK
// =====

FUNGSI BuildCandidate(heading, distance):

```

```
normalizedHeading ← normalisasi heading
```

```
projectedX ←  $X + \cos(\text{heading}) \times \text{distance}$ 
```

```
projectedY ←  $Y + \sin(\text{heading}) \times \text{distance}$ 
```

```
safeX ← batasi projectedX agar tidak keluar arena
```

```
safeY ← batasi projectedY agar tidak keluar arena
```

```
RETURN CandidateAction(
```

```
normalizedHeading,
```

```
distance,
```

```
safeX,
```

```
safeY
```

```
)
```

```
// =====
```

```
// WALL SAFETY
```

```
// =====
```

```
FUNGSI WallSafety(x, y):
```

```
left ←  $x - \text{WallMargin}$ 
```

```
right ←  $\text{ArenaWidth} - \text{WallMargin} - x$ 
```

```
bottom ←  $y - \text{WallMargin}$ 
```

```
top ←  $\text{ArenaHeight} - \text{WallMargin} - y$ 
```

```
RETURN maksimum(0, minimum(left, right, bottom, top))
```

```
// =====
```

```
// CEK TARGET VALID
```

```
// =====
```

```
FUNGSI HasFreshTarget():
```

```
RETURN
```

```
target tidak null
```

```
DAN  $\text{TurnNumber} - \text{target.lastSeenTurn} \leq \text{TargetStaleTurns}$ 
```

```
// =====
```

```
// SEARCH PATTERN
```

```
// =====
```

```
FUNGSI ExecuteSearchPattern():
```

```
centerBearing  $\leftarrow$  BearingTo(tengah arena)
```

```
nearWall  $\leftarrow$  WallSafety(X, Y) < 38
```

```
IF nearWall THEN
```

```
TurnRate  $\leftarrow$  centerBearing
```

```
MoveForward(160)
```

```
ELSE
```

```
TurnRate  $\leftarrow$  5  $\times$  moveFlip
```

```
MoveForward(110)
```

```
ENDIF
```

```
GunTurnRate  $\leftarrow$  14  $\times$  moveFlip
```

```
ExecuteWideRadarSweep()
```

```
// =====
```

```
// EVENT: MUSUH TERDETEKSI
```

```
// =====
```

```
FUNGSI OnScannedBot(event):
```

```
UpdateTarget(event)
```

```
IF target tidak null DAN target.id = event.id THEN
```

```
fullScanMode  $\leftarrow$  false
```

```
QuickLockRadar(target)
```

```
gunBearing  $\leftarrow$  GunBearingTo(target)
```

```
GunTurnRate  $\leftarrow$  gunBearing
```

```
IF data target sangat baru
```

```
DAN |gunBearing| < 6
```

```
DAN GunHeat = 0
```

```
DAN Energy > 1.2
```

```
THEN
```

```
SetFire(minimum(2.2, maksimum(0.5, Energy - 0.2)))
```

```
ENDIF
```

```

ELSE IF target = null THEN
fullScanMode ← false
ENDIF

// =====
// UPDATE TARGET
// =====

FUNGSI UpdateTarget(event):

scannedDistance ← jarak bot ke musuh yang discan

scannedScore ← ComputeTargetPriority(
scannedDistance,
event.energy,
apakah musuh ini target lama
)

IF target = null THEN
target ← EnemySnapshot dari event
RETURN
ENDIF

currentScore ← ComputeTargetPriority(
jarak ke target saat ini,
target.energy,
true
)

targetAge ← TurnNumber - target.lastSeenTurn

IF event.id = target.id THEN
target ← update data target dari event
RETURN
ENDIF

IF targetAge > 2
ATAU scannedScore >= currentScore + 8
THEN
target ← EnemySnapshot dari event
ENDIF

```

```
// =====  
// SKOR PRIORITAS TARGET  
// =====
```

FUNGSI ComputeTargetPriority(distance, enemyEnergy, alreadyLocked):

IF alreadyLocked THEN

lockBonus $\leftarrow$ 90

ELSE

lockBonus $\leftarrow$ 0

ENDIF

RETURN

2600 / (distance + 40)

+ (140 - enemyEnergy)

+ lockBonus

```
// =====  
// GREEDY AIM DAN FIRE  
// =====
```

FUNGSI ExecuteGreedyAimAndFire(enemy):

fireOptions $\leftarrow$ {0.8, 1.3, 1.9, 2.5, 3.0}

bestScore $\leftarrow -\infty$

bestFirepower $\leftarrow$ 1.0

bestAimX $\leftarrow$ enemy.x

bestAimY $\leftarrow$ enemy.y

UNTUK setiap firepower dalam fireOptions:

IF Energy $\leq$ firepower + 0.3 THEN

lanjut ke firepower berikutnya

ENDIF

predicted $\leftarrow$ PredictEnemyPosition(enemy, firepower)

aimWallDistance $\leftarrow$ jarak titik prediksi dari dinding terdekat


```

IF aimWallDistance < BotRadius THEN
lanjut ke firepower berikutnya
ENDIF

distance ← jarak bot ke titik prediksi
gunOffset ← |GunBearingTo(titik prediksi)|

hitChance ← EstimateHitChance(distance, gunOffset, enemy.speed)

IF aimWallDistance < WallFireMargin THEN
hitChance ← hitChance × faktor_penalti_dekat_dinding
ENDIF

damage ← BulletDamage(firepower)
energyPenalty ← firepower × faktor_kondisi_energi

score ← damage × hitChance - energyPenalty

IF score > bestScore THEN
bestScore ← score
bestFirepower ← firepower
bestAimX ← predicted.x
bestAimY ← predicted.y
ENDIF

ENDFOR

gunBearing ← GunBearingTo(bestAimX, bestAimY)
GunTurnRate ← gunBearing

IF data target terlalu lama THEN
RETURN
ENDIF

IF fullScanMode THEN
RETURN
ENDIF

IF GunHeat <= 0.1
DAN |gunBearing| <= 9
THEN

```

```
SetFire(minimum(bestFirepower, maksimum(0.3, Energy - 0.3)))  
ENDIF
```

```
// =====  
// PREDIKSI POSISI MUSUH  
// =====
```

```
FUNGSI PredictEnemyPosition(enemy, firepower):
```

```
bulletSpeed  $\leftarrow$  20 - 3  $\times$  firepower
```

```
predictedX  $\leftarrow$  enemy.x
```

```
predictedY  $\leftarrow$  enemy.y
```

```
ULANGI 5 kali:
```

```
distance  $\leftarrow$  jarak bot ke titik prediksi
```

```
time  $\leftarrow$  distance / bulletSpeed
```

```
predictedX  $\leftarrow$  enemy.x + enemy.velocityX  $\times$  time
```

```
predictedY  $\leftarrow$  enemy.y + enemy.velocityY  $\times$  time
```

```
batasi predictedX dan predictedY agar tetap dalam arena
```

```
ENDULANGI
```

```
RETURN Point(predictedX, predictedY)
```

```
// =====  
// ESTIMASI HIT CHANCE  
// =====
```

```
FUNGSI EstimateHitChance(distance, gunOffset, enemySpeed):
```

```
distanceFactor  $\leftarrow$  batasi(1.2 - distance / 620, 0.15, 1.0)
```

```
aimFactor  $\leftarrow$  batasi(1 - gunOffset / 42, 0.08, 1.0)
```

```
speedFactor  $\leftarrow$  batasi(1 - |enemySpeed| / 11, 0.25, 1.0)
```

```
RETURN
```

```
batasi(distanceFactor  $\times$  aimFactor  $\times$  speedFactor, 0.05, 0.97)
```

```

// =====
// RADAR LOCK
// =====

FUNGSI LockRadar(enemy):

    radarBearing ← RadarBearingTo(enemy)

    turn ← radarBearing × 2.0

    IF |turn| < 3 THEN
        turn ← 3 × arah radarBearing
    ENDIF

    turn ← batasi turn
    RadarTurnRate ← turn

    IF |radarBearing| < 1.5 THEN
        Rescan()
    ENDIF

// =====
// QUICK LOCK RADAR
// =====

FUNGSI QuickLockRadar(enemy):

    radarBearing ← RadarBearingTo(enemy)
    turn ← radarBearing × 2.4
    turn ← ClampRadarTurn(turn)

    RadarTurnRate ← turn

// =====
// WIDE RADAR SWEEP
// =====

FUNGSI ExecuteWideRadarSweep():

    IF |RadarTurnRemaining| < 1 THEN
        radarSweepDirection ← -radarSweepDirection

```

```
RadarTurnRate  $\leftarrow$   $360 \times \text{radarSweepDirection}$   
ENDIF
```

```
// =====  
// EVENT: TERKENA PELURU  
// =====
```

```
FUNGSI OnHitByBullet(event):
```

```
moveFlip  $\leftarrow$  -moveFlip  
velocityMode  $\leftarrow$  -velocityMode  
lastDirectionSwitchTurn  $\leftarrow$  TurnNumber
```

```
TurnRate  $\leftarrow$  nilai acak antara 6 sampai  $10 \times \text{moveFlip}$ 
```

```
IF peluang acak terpenuhi THEN  
MoveForward(170)  
ELSE  
MoveBack(170)  
ENDIF
```

```
fullScanMode  $\leftarrow$  true  
fullScanStartTurn  $\leftarrow$  TurnNumber
```

```
// =====  
// EVENT: MENABRAK DINDING  
// =====
```

```
FUNGSI OnHitWall(event):
```

```
moveFlip  $\leftarrow$  -moveFlip  
velocityMode  $\leftarrow$  -velocityMode  
velocityTurnCounter  $\leftarrow$  0
```

```
MoveBack(150)  
TurnRate  $\leftarrow$   $48 \times \text{moveFlip}$ 
```

```
// =====  
// EVENT: MENABRAK BOT  
// =====
```

FUNGSI OnHitBot(event):

moveFlip $\leftarrow$ -moveFlip

MoveBack(90)

TurnRate $\leftarrow$ BearingTo(event.position) + $30 \times$ moveFlip

IF GunHeat $\leq$ 0.1 THEN

SetFire(minimum(2.5, maksimum(0.6, Energy - 0.4)))

ENDIF

// =====

// EVENT: PELURU MENGENAI DINDING

// =====

FUNGSI OnBulletHitWall(event):

fullScanMode $\leftarrow$ true

fullScanStartTurn $\leftarrow$ TurnNumber

// =====

// EVENT: PELURU MENGENAI BOT

// =====

FUNGSI OnBulletHit(event):

Rescan()

// =====

// EVENT: BOT MUSUH MATI

// =====

FUNGSI OnBotDeath(event):

IF target.id = bot yang mati THEN

target $\leftarrow$ null

wasTrackingTarget $\leftarrow$ false

BeginFullScan()

4.1.4 Pseudocode Alternative Bot 3

Pseudocode **Gryffin** menunjukkan bahwa bot menerapkan algoritma greedy pada beberapa tahap pengambilan keputusan, yaitu pemilihan mode taktis, pemilihan jalur gerak, pemilihan target, dan pemilihan firepower. Pada setiap turn, bot membangun beberapa kandidat aksi, menghitung skor untuk masing-masing kandidat berdasarkan heuristik tertentu, kemudian memilih kandidat dengan skor tertinggi sebagai aksi terbaik pada kondisi saat itu.

```
{ ===== }  
{ INISIALISASI KONSTANTA DAN VARIABEL }  
{ ===== }
```

KONSTANTA:

SafeMargin $\leftarrow$ 56

WallSoftMargin $\leftarrow$ 124

CriticalEnergy $\leftarrow$ 12

LowEnergy $\leftarrow$ 20

RecoverEnergy $\leftarrow$ 34

RecentLocCount $\leftarrow$ 42

RecentCellCount $\leftarrow$ 30

BaseTravel $\leftarrow$ 138

DuelTravel $\leftarrow$ 160

MeleeTravel $\leftarrow$ 150

VARIABEL:

enemies $\leftarrow$ dictionary data musuh

recentLocations $\leftarrow$ daftar posisi bot sebelumnya

visitedCells $\leftarrow$ daftar cell arena yang pernah dikunjungi

visitedCellOrder $\leftarrow$ queue untuk membatasi cell lama

moveHoldTurns $\leftarrow$ 0

lastMoveHeading $\leftarrow$ 0

lastTargetId $\leftarrow$ -1

lastOrbitFlipTurn $\leftarrow$ -999

lastModeShiftTurn $\leftarrow$ -999

```

panicUntil ← 0
pressUntil ← 0
forceCenterUntil ← 0

radarSweepDir ← 1
orbitSign ← 1

mode ← Stabilize

{ ===== }
{ PROGRAM UTAMA }
{ ===== }

PROCEDURE Run
  Atur warna bot
  Pisahkan rotasi badan, gun, dan radar
  MaxSpeed ← 8

  WHILE bot masih berjalan DO
    PurgeDeadEnemies()

    Putar radar 360 derajat

    RefreshTacticalMode()

    ExecuteMovement()

    target ← SelectTarget()

    AimGun(target)

  Go()
END WHILE

```

```
{ ===== }  
{ PEMILIHAN MODE TAKTIS }  
{ ===== }
```

PROCEDURE RefreshTacticalMode

alive $\leftarrow$ jumlah musuh yang masih hidup

lowEnergy $\leftarrow$ Energy < LowEnergy

critical $\leftarrow$ Energy $\leq$ CriticalEnergy

underPressure $\leftarrow$ TurnNumber < panicUntil

forcedCenter $\leftarrow$ TurnNumber < forceCenterUntil

IF critical THEN

nextMode $\leftarrow$ Recover

ELSE IF forcedCenter OR IsNearBoundary(SafeMargin) THEN

nextMode $\leftarrow$ Stabilize

ELSE IF underPressure THEN

nextMode $\leftarrow$ BreakLine

ELSE IF alive $\geq$ 3 THEN

nextMode $\leftarrow$ Pressure

ELSE IF lowEnergy THEN

nextMode $\leftarrow$ Hunt

ELSE

nextMode $\leftarrow$ Hunt

END IF

IF nextMode $\neq$ mode THEN

lastModeShiftTurn $\leftarrow$ TurnNumber

END IF

mode $\leftarrow$ nextMode


```
{ ===== }  
{ MOVEMENT UTAMA BERBASIS GREEDY LANE }  
{ ===== }
```

PROCEDURE ExecuteMovement

centerX $\leftarrow$ ArenaWidth / 2

centerY $\leftarrow$ ArenaHeight / 2

IF moveHoldTurns > 0 THEN

moveHoldTurns $\leftarrow$ moveHoldTurns - 1

DriveInDirection(lastMoveHeading, 8.0)

RETURN

END IF

IF IsNearBoundary(SafeMargin) OR IsEnergyCritical() THEN

home $\leftarrow$ arah menuju titik tengah arena

lastMoveHeading $\leftarrow$ home

moveHoldTurns $\leftarrow$ 1

IF IsEnergyCritical() THEN

speed $\leftarrow$ 8.0

ELSE

speed $\leftarrow$ 7.0

END IF

DriveInDirection(home, speed)

RETURN

END IF

UpdateRecentLocations()

enemiesAlive $\leftarrow$ AliveEnemies()

```

    roam ← NormalizeAbsoluteAngle(Direction + 37 * orbitSign)
    lastMoveHeading ← roam
    moveHoldTurns ← 1
    DriveInDirection(roam, 4.5)
    RETURN
END IF

primaryThreat ← SelectPrimaryThreat(enemiesAlive)

candidates ← BuildMovementCandidates(
    enemiesAlive,
    primaryThreat,
    centerX,
    centerY
)

bestScore ←  $-\infty$ 
bestCandidate ← kandidat pertama

FOR setiap candidate dalam candidates DO
    score ← ScoreLane(candidate, enemiesAlive, primaryThreat, centerX, centerY)

    IF score > bestScore THEN
        bestScore ← score
        bestCandidate ← candidate
    END IF
END FOR

lastMoveHeading ← bestCandidate.Heading

IF bestScore > 28 THEN
    moveHoldTurns ← 3
ELSE IF bestScore > 18 THEN
    moveHoldTurns ← 2
ELSE

```

END IF

DriveInDirection(bestCandidate.Heading, 8.0 * bestCandidate.SpeedFactor)

END PROCEDURE

```
{ ===== }  
{ MEMBANGUN KANDIDAT JALUR GERAK }  
{ ===== }
```

FUNCTION BuildMovementCandidates(enemies, primary, centerX, centerY) RETURNS list

toThreat ← arah dari bot menuju primary threat

awayThreat ← NormalizeAbsoluteAngle(toThreat + 180)

escapeCentroid ← EscapeHeadingFromCentroid(enemies)

centerHeading ← arah menuju tengah arena

angularPush ← 92 * orbitSign

shallowPush ← 54 * orbitSign

cutPush ← 128 * orbitSign

candidates ← list kosong

Tambahkan kandidat:

BuildLane(awayThreat, BaseTravel, 1.00)

BuildLane(toThreat + angularPush, DuelTravel, 1.00)

BuildLane(toThreat - angularPush, DuelTravel, 1.00)

BuildLane(toThreat + shallowPush, BaseTravel * 0.92, 0.97)

BuildLane(toThreat - shallowPush, BaseTravel * 0.92, 0.97)

BuildLane(escapeCentroid, MeleeTravel, 0.94)

BuildLane(centerHeading, BaseTravel * 0.88, 0.88)

BuildLane(Direction + 28, BaseTravel * 0.74, 0.82)

BuildLane(Direction - 28, BaseTravel * 0.74, 0.82)

BuildLane(toThreat + cutPush, BaseTravel * 0.98, 0.90)

BuildLane(toThreat - cutPush, BaseTravel * 0.98, 0.90)

BuildLane(awayThreat + 35 * orbitSign, BaseTravel * 0.82, 0.87)

```
IF mode = Pressure THEN
    Tambahkan kandidat tekanan tambahan
END IF
```

```
IF mode = Recover THEN
    Tambahkan kandidat menuju tengah arena dengan speed lebih rendah
END IF
```

```
RETURN candidates
END FUNCTION
```

```
{ ===== }
{ MEMBUAT SATU KANDIDAT JALUR }
{ ===== }
```

```
FUNCTION BuildLane(heading, distance, speedFactor) RETURNS LaneCandidate
    normalizedHeading ← NormalizeAbsoluteAngle(heading)
    projectedPoint ← ProjectStep(normalizedHeading, distance)
```

```
    RETURN LaneCandidate(
        Heading = normalizedHeading,
        SpeedFactor = speedFactor,
        X = projectedPoint.X,
        Y = projectedPoint.Y
    )
END FUNCTION
```

```
{ ===== }
{ PROYEKSI POSISI BERDASARKAN ARAH GERAK }
{ ===== }
```

```
FUNCTION ProjectStep(heading, distance) RETURNS point
    rad ← ToRadians(heading)
```

```
x ← X + cos(rad) * distance
```

```
y ← Y + sin(rad) * distance
```

```
x ← Clamp(x, batas kiri arena, batas kanan arena)
```

```
y ← Clamp(y, batas bawah arena, batas atas arena)
```

```
RETURN point(x, y)
```

```
END FUNCTION
```

```
{ ===== }
```

```
{ PENILAIAN KANDIDAT JALUR }
```

```
{ ===== }
```

```
FUNCTION ScoreLane(candidate, enemies, primary, centerX, centerY) RETURNS real
```

```
IF candidate berada di luar arena THEN
```

```
RETURN -1.000.000
```

```
END IF
```

```
minEnemy ←  $\infty$ 
```

```
weightedEnemyField ← 0
```

```
centroidX ← 0
```

```
centroidY ← 0
```

```
weightSum ← 0
```

```
FOR setiap enemy dalam enemies DO
```

```
distance ← jarak candidate ke enemy
```

```
minEnemy ← nilai minimum antara minEnemy dan distance
```

```
weightedEnemyField ← weightedEnemyField
```

```
+ bobot energi enemy / distance2
```

```
weight ← nilai ancaman enemy / distance
```

```
centroidY  $\leftarrow$  centroidY + enemy.Y * weight  
weightSum  $\leftarrow$  weightSum + weight  
END FOR
```

```
IF weightSum > 0 THEN  
centroidX  $\leftarrow$  centroidX / weightSum  
centroidY  $\leftarrow$  centroidY / weightSum  
ELSE  
centroidX  $\leftarrow$  primary.X  
centroidY  $\leftarrow$  primary.Y  
END IF
```

```
wallSafety  $\leftarrow$  WallSafety(candidate.X, candidate.Y)  
wallScore  $\leftarrow$  skor keamanan dari dinding
```

```
targetDistance  $\leftarrow$  jarak candidate ke primary threat
```

```
IF mode = Pressure THEN  
idealDistance  $\leftarrow$  245  
ELSE IF Energy > 40 THEN  
idealDistance  $\leftarrow$  280  
ELSE IF Energy > 20 THEN  
idealDistance  $\leftarrow$  345  
ELSE  
idealDistance  $\leftarrow$  420  
END IF
```

```
ringScore  $\leftarrow$  skor kedekatan terhadap jarak ideal
```

```
centroidEscape  $\leftarrow$  jarak candidate dari centroid musuh  
centroidScore  $\leftarrow$  skor menjauh dari pusat ancaman
```

```
lateralScore  $\leftarrow$  skor gerakan lateral terhadap primary threat
```

```
novelty  $\leftarrow$  skor variasi arah agar tidak terlalu repetitif
```

revisitPenalty $\leftarrow$ penalti jika candidate dekat dengan lokasi lama

cellPenalty $\leftarrow$ VisitedCellPenalty(candidate.X, candidate.Y)

IF minEnemy < 120 THEN

closeDanger $\leftarrow$ penalti karena terlalu dekat dengan musuh

ELSE

closeDanger $\leftarrow$ 0

END IF

pressure $\leftarrow$ tekanan total dari musuh

centerPull $\leftarrow$ skor berdasarkan posisi candidate terhadap tengah arena

score $\leftarrow$ wallScore * 3.2

+ ringScore * 2.2

+ centroidScore * 1.6

+ lateralScore * 1.5

+ centerPull * 0.8

+ novelty * 0.7

- revisitPenalty * 1.4

- cellPenalty * 1.25

- pressure * 0.7

+ closeDanger

IF candidate.SpeedFactor < 0.9 AND Energy < RecoverEnergy THEN

score $\leftarrow$ score + 4.0

END IF

IF IsEnemyAlignedWithPath(candidate.X, candidate.Y, primary) THEN

score $\leftarrow$ score - 16.0

END IF

IF targetDistance < 160 AND NOT ShouldRam(primary) THEN

score $\leftarrow$ score - 24.0

```
IF candidate.SpeedFactor > 0.95 AND mode = Pressure THEN
```

```
score  $\leftarrow$  score + 3.0
```

```
END IF
```

```
IF mode = BreakLine THEN
```

```
score  $\leftarrow$  score + bonus gerak memotong garis tembak
```

```
END IF
```

```
RETURN score
```

```
END FUNCTION
```

```
{ ===== }
```

```
{ MENJALANKAN BOT KE ARAH TERTENTU }
```

```
{ ===== }
```

```
PROCEDURE DriveInDirection(heading, speed)
```

```
turn  $\leftarrow$  NormalizeRelativeAngle(heading - Direction)
```

```
IF |turn| > 90 THEN
```

```
backTurn  $\leftarrow$  NormalizeRelativeAngle(turn + 180)
```

```
SetTurnLeft(backTurn)
```

```
TargetSpeed  $\leftarrow$  -|speed|
```

```
ELSE
```

```
SetTurnLeft(turn)
```

```
TargetSpeed  $\leftarrow$  |speed|
```

```
END IF
```

```
END PROCEDURE
```

```
{ ===== }
```

```
{ UPDATE RIWAYAT POSISI }
```

```
{ ===== }
```



```
IF TurnNumber bukan kelipatan 2 THEN
RETURN
END IF
```

Tambahkan posisi saat ini ke recentLocations

```
WHILE jumlah recentLocations > RecentLocCount DO
Hapus lokasi paling lama
END WHILE
```

cell ← cell arena berdasarkan posisi X dan Y

```
IF cell belum ada di visitedCells THEN
Tambahkan cell ke visitedCells
Masukkan cell ke visitedCellOrder
END IF
```

```
WHILE jumlah visitedCellOrder > RecentCellCount DO
oldCell ← keluarkan cell paling lama
Hapus oldCell dari visitedCells
END WHILE
END PROCEDURE
```

```
{ ===== }
{ PEMILIHAN TARGET GREEDY }
{ ===== }
```

```
FUNCTION SelectTarget RETURNS EnemyInfo
bestTarget ← null
bestScore ← -∞
```

```
FOR setiap enemy dalam enemies DO
IF enemy tidak hidup THEN
CONTINUE
```

```
age ← TurnNumber - enemy.LastSeenTurn
```

```
IF age > 30 THEN
```

```
CONTINUE
```

```
END IF
```

```
score ← ScoreTarget(enemy, age)
```

```
IF score > bestScore THEN
```

```
bestScore ← score
```

```
bestTarget ← enemy
```

```
END IF
```

```
END FOR
```

```
IF bestTarget tidak null THEN
```

```
lastTargetId ← bestTarget.Id
```

```
END IF
```

```
RETURN bestTarget
```

```
END FUNCTION
```

```
{ ===== }
```

```
{ SKOR TARGET }
```

```
{ ===== }
```

```
FUNCTION ScoreTarget(enemy, age) RETURNS real
```

```
distance ← jarak bot ke enemy
```

```
distanceScore ← semakin dekat semakin besar
```

```
energyScore ← semakin rendah energi enemy semakin besar
```

```
damageScore ← semakin tinggi damageFactor semakin besar
```

```
instabilityScore ← semakin tidak stabil gerak enemy semakin besar
```

```
predictabilityScore ← semakin mudah diprediksi semakin besar
```

```
IF enemy.Energy <= 15 THEN
finishBonus ← 0.7
ELSE
finishBonus ← 0
END IF
```

```
IF enemy.Id = lastTargetId THEN
continuity ← 0.14
ELSE
continuity ← 0
END IF
```

```
score ← distanceScore * 2.15
+ energyScore * 2.0
+ damageScore * 1.7
+ predictabilityScore * 1.9
+ instabilityScore * 0.9
+ freshnessScore * 0.85
+ finishBonus
+ continuity
```

```
RETURN score
END FUNCTION
```

```
{ ===== }
{ PEMILIHAN FIREPOWER }
{ ===== }
```

```
FUNCTION ChooseFirePower(target, distance) RETURNS real
IF IsEnergyCritical() THEN
RETURN 0
END IF
```

```
power ← 3.0
ELSE IF distance < 210 THEN
power ← 2.5
ELSE IF distance < 360 THEN
power ← 1.9
ELSE IF distance < 520 THEN
power ← 1.3
ELSE IF distance < 700 THEN
power ← 0.85
ELSE
power ← 0.55
END IF
```

```
IF target.Energy <= 12 THEN
power ← maksimum(power, 2.8)
END IF
```

```
alive ← jumlah musuh hidup
```

```
IF alive >= 4 THEN
power ← minimum(power, 1.6)
ELSE IF alive = 3 THEN
power ← minimum(power, 2.05)
ELSE IF alive = 2 THEN
power ← minimum(power, 2.35)
END IF
```

```
IF Energy < LowEnergy THEN
power ← minimum(power, 1.05)
END IF
```

```
IF Energy < CriticalEnergy THEN
power ← 0
END IF
```

END FUNCTION

```
{ ===== }  
{ AIMING DAN FIRING }  
{ ===== }
```

PROCEDURE AimGun(target)

IF target = null OR GunHeat > 0.1 THEN

SetFire(0)

RETURN

END IF

distance $\leftarrow$ jarak bot ke target

firepower $\leftarrow$ ChooseFirePower(target, distance)

IF firepower $\leq$ 0 THEN

SetFire(0)

RETURN

END IF

aimPoint $\leftarrow$ PredictPosition(target, firepower)

gunTurn $\leftarrow$ NormalizeRelativeAngle(
arah menuju aimPoint - GunDirection
)

SetTurnGunLeft(gunTurn)

IF |gunTurn| < 8 AND GunHeat = 0 THEN

SetFire(firepower)

ELSE

SetFire(0)

END IF

END PROCEDURE

```
{ ===== }  
{ PREDIKSI POSISI MUSUH }  
{ ===== }
```

FUNCTION PredictPosition(enemy, firepower) RETURNS AimPoint

bulletSpeed $\leftarrow$ CalcBulletSpeed(firepower)

ticks $\leftarrow$ Clamp(jarak bot ke enemy / bulletSpeed, 1, 15)

measuredVelocityX $\leftarrow$ cos(enemy.Direction) * enemy.Speed

measuredVelocityY $\leftarrow$ sin(enemy.Direction) * enemy.Speed

stability $\leftarrow$ nilai kestabilan berdasarkan enemy.DirectionVar

blend $\leftarrow$ gabungan antara velocity terukur dan velocity arah

turnBias $\leftarrow$ batas perubahan arah enemy

predictedX $\leftarrow$ enemy.X

predictedY $\leftarrow$ enemy.Y

FOR i $\leftarrow$ 1 TO 4 DO

predictedHeading $\leftarrow$ enemy.Direction + turnBias * ticks * blend

accelerationX $\leftarrow$ cos(predictedHeading) * enemy.Speed * ticks

accelerationY $\leftarrow$ sin(predictedHeading) * enemy.Speed * ticks

velocityX $\leftarrow$ enemy.VelocityX * (1 - blend) + measuredVelocityX * blend

velocityY $\leftarrow$ enemy.VelocityY * (1 - blend) + measuredVelocityY * blend

predictedX $\leftarrow$ enemy.X + kombinasi velocityX dan accelerationX

predictedY $\leftarrow$ enemy.Y + kombinasi velocityY dan accelerationY

IF predictedX atau predictedY keluar arena THEN

predictedX $\leftarrow$ Clamp(predictedX, batas arena)

predictedY $\leftarrow$ Clamp(predictedY, batas arena)

END IF

ticks $\leftarrow$ jarak bot ke titik prediksi / bulletSpeed

END FOR

RETURN AimPoint(predictedX, predictedY)

END FUNCTION

```
{ ===== }  
{ EVENT: RADAR MENDETEKSI MUSUH }  
{ ===== }
```

PROCEDURE OnScannedBot(event)

IF event adalah teammate THEN

RETURN

END IF

enemy $\leftarrow$ GetOrCreate(event.ScannedBotId)

Simpan posisi, arah, dan speed sebelumnya

IF enemy sudah memiliki sampel sebelumnya THEN

Hitung velocity berdasarkan perpindahan posisi

Hitung velocity berdasarkan arah dan speed

Gabungkan kedua velocity

ELSE

Hitung velocity berdasarkan arah dan speed

Tandai enemy sudah memiliki sampel

END IF

Perbarui data enemy:

X, Y, Energy, Direction, Speed, LastSeenTurn, Alive

Tambah ScanCount, TotalDistance, dan TurnsTracked

Hitung perubahan arah enemy

Hitung perubahan speed enemy

Update DirectionVar dan SpeedVar

Hitung ThreatIndex berdasarkan:

energi enemy

jarak enemy

instabilitas gerakan

Hitung DamageFactor berdasarkan:

damage yang pernah diterima

rata-rata jarak

kestabilan gerak enemy

Hitung PressureIndex

Hitung FinishIndex

IF PressureIndex tinggi AND jarak enemy dekat THEN

panicUntil $\leftarrow$ TurnNumber + 9

forceCenterUntil $\leftarrow$ TurnNumber + 7

END IF

END PROCEDURE

{ ===== }

{ EVENT: BOT MUSUH MATI }

{ ===== }

PROCEDURE OnBotDeath(event)

IF enemy dengan ID event.VictimId ada THEN

enemy.Alive $\leftarrow$ false

END IF

END PROCEDURE


```
{ ===== }  
{ EVENT: MENABRAK DINDING }  
{ ===== }
```

```
PROCEDURE OnHitWall(event)  
  moveHoldTurns ← 2  
  forceCenterUntil ← TurnNumber + 12  
  orbitSign ← -orbitSign  
  lastOrbitFlipTurn ← TurnNumber  
  lastMoveHeading ← arah menuju tengah arena  
  
  TargetSpeed ← -8  
  SetTurnLeft(lastMoveHeading - Direction)  
END PROCEDURE
```

```
{ ===== }  
{ EVENT: MENABRAK BOT LAIN }  
{ ===== }
```

```
PROCEDURE OnHitBot(event)  
  moveHoldTurns ← 1  
  orbitSign ← -orbitSign  
  lastOrbitFlipTurn ← TurnNumber  
  
  away ← arah menjauh dari bot yang ditabrak  
  
  lastMoveHeading ← away  
  
  SetTurnLeft(away - Direction)  
  TargetSpeed ← -8  
END PROCEDURE
```

```
{ ===== }
```

```
{ ===== }
```

```
PROCEDURE OnHitByBullet(event)
```

```
shooterId ← ID pemilik peluru
```

```
IF shooterId ada dalam enemies THEN
```

```
shooter ← enemies[shooterId]
```

```
damage ← damage peluru berdasarkan power
```

```
shooter.DamageTaken ← shooter.DamageTaken + damage
```

```
shooter.DamageFactor ← update berdasarkan damage dan jarak rata-rata
```

```
shooter.ThreatIndex ← shooter.ThreatIndex + 0.25
```

```
shooter.PressureIndex ← shooter.PressureIndex + 0.2
```

```
END IF
```

```
panicUntil ← TurnNumber + 10
```

```
moveHoldTurns ← maksimum(moveHoldTurns, 1)
```

```
orbitSign ← -orbitSign
```

```
END PROCEDURE
```

```
{ ===== }
```

```
{ EVENT: BOT MATI }
```

```
{ ===== }
```

```
PROCEDURE OnDeath(event)
```

```
Hapus semua data enemies
```

```
Hapus semua recentLocations
```

```
Hapus semua visitedCells
```

```
Hapus semua visitedCellOrder
```

```
END PROCEDURE
```

```
{ ===== }
```

```
{ ===== }
```

```
FUNCTION GetOrCreate(id) RETURNS EnemyInfo
```

```
IF id belum ada dalam enemies THEN
```

```
  Buat EnemyInfo baru
```

```
  Simpan ke enemies
```

```
END IF
```

```
RETURN enemies[id]
```

```
END FUNCTION
```

```
FUNCTION AliveEnemies RETURNS list
```

```
RETURN semua enemy dalam enemies yang Alive = true
```

```
END FUNCTION
```

```
PROCEDURE PurgeDeadEnemies
```

```
FOR setiap enemy hidup dalam enemies DO
```

```
IF TurnNumber - enemy.LastSeenTurn > 42 THEN
```

```
  enemy.Alive ← false
```

```
END IF
```

```
END FOR
```

```
END PROCEDURE
```

```
FUNCTION IsEnergyCritical RETURNS boolean
```

```
RETURN Energy <= CriticalEnergy
```

```
END FUNCTION
```

```
FUNCTION ShouldRam(target) RETURNS boolean
```

```
RETURN Energy > 26
```

```
AND target.Energy <= 18
```

```
AND jarak bot ke target < 175
```

FUNCTION SelectPrimaryThreat(enemies) RETURNS EnemyInfo

best $\leftarrow$ musuh pertama

bestScore $\leftarrow -\infty$

FOR setiap enemy dalam enemies DO

distance $\leftarrow$ jarak bot ke enemy

bodyBearing $\leftarrow$ sudut enemy terhadap arah badan bot

closeFactor $\leftarrow$ semakin dekat semakin besar

energyFactor $\leftarrow$ berdasarkan energi enemy

flankFactor $\leftarrow$ bonus jika enemy berada di samping

backFactor $\leftarrow$ bonus jika enemy berada di belakang

agePenalty $\leftarrow$ penalti jika data enemy sudah lama

score $\leftarrow$ closeFactor * faktor ancaman

+ enemy.PressureIndex

- agePenalty

IF distance < 170 THEN

score $\leftarrow$ score + 1

END IF

IF enemy.Id = lastTargetId THEN

score $\leftarrow$ score - 0.08

END IF

IF score > bestScore THEN

bestScore $\leftarrow$ score

best $\leftarrow$ enemy

END IF

END FOR

RETURN best

4.2 Struktur Data, Fungsi, dan Prosedur Bot Utama

4.2.1. Struktur Data (Penyimpanan Informasi)

Bot TankAlbino menggunakan beberapa struktur data utama untuk menyimpan informasi musuh, hasil prediksi, serta state permainan selama pertandingan berlangsung.

Struktur Data	Fungsi
Dictionary<int, Enemy> enemies	Menyimpan seluruh data musuh berdasarkan ID musuh
Enemy	Menyimpan informasi posisi, energi, arah, kecepatan, acceleration, dan status musuh
AimPoint	Menyimpan titik koordinat hasil prediksi target
orbitSign	Menentukan arah orbit movement bot
radarSign	Menentukan arah putaran radar
lastTargetId	Menyimpan target terakhir agar bot tidak terlalu sering mengganti target
duelJinkUntil	Penanda durasi mode jink setelah terkena peluru
forceCenterUntil	Penanda durasi paksa kembali ke tengah arena
lastBulletHitTurn	Menyimpan tick terakhir saat bot terkena peluru
lastReverseTurn	Menyimpan tick terakhir perubahan arah orbit

Penjelasan Struktur Data

- **Dictionary Enemy**

Dictionary digunakan karena bot harus mengakses data musuh secara cepat berdasarkan ID. Struktur ini memungkinkan proses update data radar dan pencarian target greedy dilakukan secara efisien.

- **Struktur Enemy**

Objek Enemy menyimpan:

- posisi musuh,

- energi musuh,
- velocity,
- acceleration,
- lateral movement,
- dan timestamp scan terakhir.

Data ini digunakan untuk:

- target scoring,
- predictive targeting,
- threat analysis,
- dan melee movement scoring.

- **AimPoint**

AimPoint digunakan untuk menyimpan koordinat hasil prediksi posisi musuh yang nantinya digunakan sebagai titik bidik peluru.

4.2.2. Fungsi (Kalkulasi & Menghasilkan Nilai)

Bot TankAlbino terdiri dari beberapa fungsi utama yang saling bekerja sama untuk mengimplementasikan strategi greedy.

4.2.3. Prosedur (Eksekusi Aksi & Merubah State)

Fungsi / Prosedur	Keterangan
Run()	Loop utama bot
SelectGreedyTarget()	Memilih target terbaik menggunakan scoring greedy
GreedyTargetScore()	Menghitung skor prioritas target
ControlRadar()	Mengatur radar lock dan radar sweep
ControlGunAndFire()	Mengatur gun dan penembakan
EstimateFirepower()	Menentukan firepower adaptif
SelectAimPoint()	Memilih metode prediksi target
PredictEnemyPosition()	Prediksi linear posisi musuh
PredictObservedEnemyPosition()	Prediksi observed movement untuk target zig-zag
ControlMovement()	Mengatur movement duel

ControlMeleeMovement()	Mengatur movement melee
ScoreMeleePosition()	Menghitung skor keamanan posisi
MoveToward()	Menggerakkan bot menuju heading tertentu
ShouldRam()	Menentukan apakah kondisi cocok untuk ramming
IsMeleeMode()	Mendeteksi mode melee

Penjelasan Fungsi Utama

- **Fungsi SelectGreedyTarget()**

Fungsi ini memilih target terbaik berdasarkan skor greedy tertinggi. Skor dihitung menggunakan:

- peluang hit,
- energi target,
- peluang kill,
- ancaman target,
- dan peluang ram.

Fungsi ini merupakan inti dari strategi greedy TankAlbino.

- **Fungsi ControlRadar()**

Radar dikontrol menggunakan kombinasi:

- radar lock,
- overscan,
- dan radar sweep.

Pada duel, radar fokus mengunci target aktif agar data target selalu diperbarui. Pada melee, radar melakukan sweep untuk mendeteksi ancaman dari berbagai arah.

- **Fungsi EstimateFirepower()**

Firepower ditentukan secara adaptif berdasarkan:

- jarak target,
- energi bot,
- energi target,
- dan kondisi arena.

Strategi ini bertujuan menjaga keseimbangan antara damage output dan efisiensi energi.

- **Fungsi PredictEnemyPosition()**

Fungsi ini melakukan prediksi linear menggunakan:

- arah target,
- kecepatan target,
- dan waktu tempuh peluru.

Prediksi digunakan untuk meningkatkan akurasi tembakan.

- **Fungsi ControlMovement()**

Movement duel menggunakan:

- orbit movement,
- reverse orbit,
- adaptive angle,
- dan jink movement.

Strategi ini membuat movement bot lebih sulit diprediksi musuh.

- **Fungsi ControlMeleeMovement()**

Pada mode melee, bot mengevaluasi banyak kandidat posisi dan memilih posisi dengan skor keamanan tertinggi.

Evaluasi mempertimbangkan:

- jarak dinding,
- crossfire,
- posisi musuh,
- dan threat centroid.

4.3 Pengujian Program

4.3.1 Skenario Pengujian

Pengujian dilakukan untuk mengetahui performa bot utama yang dipilih, yaitu Ayam Gak Jago 2.0 atau **TankAlbino**, dalam pertandingan 1 vs 1 melawan bot asisten **Corners** 1.0. Pengujian dilakukan sebanyak 3 kali battle. Setiap battle terdiri dari 10 ronde.

Tujuan dari pengujian ini adalah untuk melihat apakah strategi **Hybrid Tactical Greedy** yang digunakan oleh **TankAlbino** mampu bekerja sesuai rancangan. Aspek yang diamati dalam pengujian meliputi kemampuan bot dalam memilih target, mempertahankan radar lock, menentukan firepower, melakukan predictive targeting, menghindari serangan lawan, serta memperoleh skor melalui bullet damage, survival score, dan last survivor bonus.

Bot lawan yang digunakan adalah Corners 1.0. Bot ini dipilih sebagai bot asisten untuk melihat kemampuan **TankAlbino** dalam menghadapi lawan dengan pola permainan yang cenderung bertahan di area sudut arena. Pola seperti ini penting untuk diuji karena dapat menunjukkan apakah bot mampu mengejar target, menjaga akurasi tembakan, dan tetap menghindari risiko wall damage atau posisi yang kurang menguntungkan.

Pengujian dilakukan dalam tiga battle sebagai berikut.

Battle 1:

Ayam Gak Jago 2.0 memperoleh total skor 1710. Skor tersebut terdiri dari survival score sebesar 500, last survivor bonus sebesar 100, bullet damage sebesar 925, bullet damage bonus sebesar 185, ram damage sebesar 0, dan ram damage bonus sebesar 0. Bot ini memperoleh peringkat pertama sebanyak 10 ronde.

Corners 1.0 memperoleh total skor 25. Skor tersebut terdiri dari bullet damage sebesar 24 dan ram damage sebesar 1. Bot Corners tidak memperoleh survival score maupun last survivor bonus, dan berada pada peringkat kedua sebanyak 10 ronde.

Battle 2:

Ayam Gak Jago 2.0 memperoleh total skor 1692. Skor tersebut terdiri dari survival score sebesar 500, last survivor bonus sebesar 100, bullet damage sebesar 905, bullet damage bonus sebesar 181, dan ram damage sebesar 6. Bot ini kembali memperoleh peringkat pertama sebanyak 10 ronde. Corners 1.0 memperoleh total skor 1. Skor tersebut berasal dari ram damage sebesar 1. Pada battle ini, Corners tidak berhasil memperoleh bullet damage, survival score, maupun last survivor bonus.

Battle 3:

Ayam Gak Jago 2.0 memperoleh total skor 1692. Skor tersebut terdiri dari survival score sebesar 500, last survivor bonus sebesar 100, bullet damage sebesar 910, bullet damage bonus sebesar 181, ram damage sebesar 0, dan ram damage bonus sebesar 0. Bot ini kembali memperoleh peringkat pertama sebanyak 10 ronde.

Corners 1.0 memperoleh total skor 4. Skor tersebut berasal dari bullet damage sebesar 4. Pada battle ini, Corners tidak memperoleh survival score, last survivor bonus, maupun ram damage.

4.3.2 Hasil Pengujian dan Analisis

Berdasarkan tiga kali pengujian yang dilakukan, bot Ayam Gak Jago 2.0 atau TankAlbino selalu berhasil memenangkan seluruh ronde melawan Corners 1.0. Pada setiap battle, TankAlbino memperoleh peringkat pertama sebanyak 10 kali dari 10 ronde. Hal ini menunjukkan bahwa strategi greedy yang diterapkan mampu menghasilkan performa yang sangat stabil dalam skenario 1 vs 1 melawan bot asisten Corners.

Rata-rata total skor TankAlbino dari tiga battle adalah 1698 poin. Skor tertinggi diperoleh pada Battle 1 dengan total 1710 poin, sedangkan Battle 2 dan Battle 3 menghasilkan skor yang sama, yaitu 1692 poin. Perbedaan skor antar battle relatif kecil, sehingga dapat disimpulkan bahwa performa bot cukup konsisten.

Dari sisi survival, TankAlbino selalu memperoleh survival score sebesar 500 dan last survivor bonus sebesar 100 pada setiap battle. Hal ini menunjukkan bahwa bot selalu mampu bertahan sampai akhir ronde dan selalu menjadi bot terakhir yang hidup. Keberhasilan ini menunjukkan bahwa movement dan strategi bertahan yang digunakan oleh TankAlbino berjalan efektif dalam skenario 1 vs 1.

Dari sisi serangan, TankAlbino menghasilkan bullet damage yang sangat tinggi. Pada Battle 1, bot menghasilkan bullet damage sebesar 925. Pada Battle 2, bullet damage sebesar 905. Pada Battle 3, bullet damage sebesar 910. Rata-rata bullet damage yang dihasilkan adalah sekitar 913,33 poin. Nilai ini menunjukkan bahwa strategi pemilihan target, predictive targeting, dan adaptive firepower dapat bekerja dengan baik. Bot mampu menembak lawan secara konsisten dan menghasilkan damage besar.

Bullet damage bonus TankAlbino juga stabil, yaitu 185 pada Battle 1 dan 181 pada Battle 2 serta Battle 3. Nilai ini menunjukkan bahwa bot tidak hanya menghasilkan damage, tetapi juga berhasil mengeliminasi lawan secara efektif. Karena pertandingan dilakukan dalam format 1 vs 1, bonus ini menunjukkan keberhasilan bot dalam menyelesaikan ronde dengan kill secara konsisten.

Komponen ram damage pada TankAlbino relatif kecil. Pada Battle 1 dan Battle 3, ram damage bernilai 0. Pada Battle 2, bot hanya memperoleh ram damage sebesar 6. Hal ini menunjukkan bahwa TankAlbino tidak terlalu bergantung pada strategi tabrakan. Strategi utama bot lebih dominan pada

bullet damage, target scoring, predictive fire, dan survival. Kondisi ini sesuai dengan rancangan TankAlbino sebagai bot utama yang seimbang, bukan bot rammer agresif.

Sementara itu, Corners 1.0 memperoleh skor yang sangat rendah pada seluruh pengujian. Pada Battle 1, Corners memperoleh total skor 25. Pada Battle 2, Corners hanya memperoleh total skor 1. Pada Battle 3, Corners memperoleh total skor 4. Rata-rata total skor Corners hanya 10 poin. Corners tidak pernah memperoleh survival score atau last survivor bonus, yang berarti bot tersebut selalu kalah sebelum akhir ronde.

Damage yang dihasilkan Corners juga sangat kecil. Pada Battle 1, Corners hanya menghasilkan bullet damage 24 dan ram damage 1. Pada Battle 2, Corners hanya menghasilkan ram damage 1. Pada Battle 3, Corners hanya menghasilkan bullet damage 4. Hal ini menunjukkan bahwa movement dan strategi serangan TankAlbino cukup efektif dalam mengurangi peluang Corners untuk memberikan damage.

Berdasarkan hasil tersebut, strategi Hybrid Tactical Greedy yang digunakan oleh TankAlbino dapat dikatakan berhasil dalam skenario pengujian ini. Bot mampu memilih keputusan lokal yang menguntungkan pada setiap turn, seperti memilih target yang valid, menjaga radar lock, menentukan firepower yang sesuai, memprediksi posisi musuh, dan menjaga posisi agar tetap aman. Kombinasi keputusan greedy tersebut menghasilkan skor yang tinggi dan konsisten.

Namun, strategi greedy ini tidak dapat dikatakan selalu optimal secara global untuk semua kondisi pertandingan. Pada pengujian ini, lawan yang digunakan adalah Corners 1.0 dalam format 1 vs 1. Strategi TankAlbino terbukti sangat efektif pada kondisi tersebut, tetapi hasilnya dapat berbeda jika melawan bot dengan movement yang lebih evasive, bot dengan pola tembakan yang lebih agresif, atau pertandingan melee dengan banyak musuh. Pada kondisi melee, bot harus mempertimbangkan lebih banyak ancaman, seperti crossfire, posisi musuh dari berbagai arah, dan risiko dikepung. Secara keseluruhan, hasil pengujian menunjukkan bahwa TankAlbino layak dipilih sebagai bot utama. Bot berhasil memperoleh skor tinggi, selalu memenangkan seluruh ronde, menghasilkan bullet damage besar, memperoleh survival score penuh, dan menjaga konsistensi performa pada tiga battle. Strategi greedy yang diterapkan berhasil mencapai tujuan utama permainan, yaitu memperoleh skor akhir setinggi mungkin. Untuk data hasilnya dapat dilihat pada 3 gambar dibawah:

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ayam Gak Jago 2.0	1710	500	100	925	185	0	0	10	0	0
2	Corners 1.0	25	0	0	24	0	1	0	0	10	0

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ayam Gak Jago 2.0	1692	500	100	905	181	6	0	10	0	0
2	Corners 1.0	1	0	0	0	0	1	0	0	10	0

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Ayam Gak Jago 2.0	1692	500	100	910	181	0	0	10	0	0
2	Corners 1.0	4	0	0	4	0	0	0	0	10	0

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Kesimpulan dari perancangan, implementasi, dan pengujian yang telah dilakukan, algoritma greedy dapat diterapkan dengan baik pada bot Robocode Tank Royale karena bot harus mengambil keputusan secara cepat pada setiap turn berdasarkan kondisi arena saat itu. Keempat bot yang dibuat memiliki pendekatan greedy dan heuristic yang berbeda, yaitu TankAlbino sebagai bot utama dengan strategi Hybrid Tactical Greedy, BantengMerah dengan strategi agresif berbasis ramming, AiranScube dengan strategi evasive movement, dan Gryffin dengan strategi tactical lane scoring. Dari hasil analisis, TankAlbino dipilih sebagai bot utama karena memiliki keseimbangan paling baik antara menyerang dan bertahan, mampu memilih target berdasarkan scoring, menggunakan predictive targetting, menyesuaikan firepower, menjaga radar lock, serta menerapkan movement adaptif untuk duel maupun melee. Strategi ini sesuai dengan fungsi objektif permainan, yaitu memperoleh skor setinggi mungkin melalui bullet damage, survival score, bonus kill, last survivor bonus, dan ram damage.

Hasil pengujian menunjukkan bahwa bot Ayam Gak Jago 2.0 atau TankAlbino berhasil memenangkan seluruh ronde dalam tiga kali battle melawan Corners 1.0. TankAlbino selalu memperoleh peringkat pertama sebanyak 10 ronde pada setiap battle, dengan rata-rata total skor 1698 poin. Bot juga selalu mendapatkan survival score sebesar 500 dan last survivor bonus sebesar 100, serta menghasilkan rata-rata bullet damage sekitar 913,33 poin. Hal ini menunjukkan bahwa strategi Hybrid Tactical Greedy berhasil berjalan stabil dalam pengujian 1 vs 1, terutama dalam pemilihan target, predictive targetting, adaptive firepower, movement bertahan, dan penyelesaian ronde secara konsisten. Namun, strategi ini belum dapat dikatakan selalu optimal untuk semua kondisi karena pengujian utama dilakukan pada skenario 1 vs 1 melawan Corners, sehingga performanya masih perlu diuji lebih lanjut pada lawan yang lebih evasive, agresif, atau pada pertandingan melee dengan banyak musuh

5.2 Saran

Saran pengembangan dari bot ini adalah Pengembangan selanjutnya dapat difokuskan pada peningkatan kemampuan bot dalam menghadapi lebih banyak variasi lawan, terutama bot dengan movement yang lebih sulit diprediksi dan bot yang agresif dalam menyerang. Meskipun TankAlbino sudah memiliki performa yang stabil pada pengujian 1 vs 1, strategi greedy yang digunakan masih dapat diperbaiki melalui tuning parameter target scoring, firepower, radar lock, dan movement agar lebih adaptif pada kondisi melee. Pengujian juga sebaiknya diperluas dengan lebih banyak jenis bot lawan dan skenario pertandingan, misalnya melawan bot evasive, bot rammer, bot dengan pola tembakan kuat, serta pertandingan banyak bot agar efektivitas strategi dapat dinilai secara lebih menyeluruh.

Selain itu, bagian prediksi tembakan dan penghindaran serangan dapat dikembangkan lebih lanjut agar bot tidak hanya kuat terhadap lawan pasif seperti Corners, tetapi juga mampu menghadapi lawan dengan perubahan arah yang ekstrem. Strategi alternatif seperti BantengMerah, AiranScube, dan Gryffin juga dapat dijadikan bahan pengembangan lanjutan, misalnya dengan mengambil agresivitas BantengMerah, movement evasive AiranScube, dan analisis posisi Gryffin untuk memperkaya strategi TankAlbino. Dengan pengujian yang lebih luas dan penyempurnaan heuristic, bot diharapkan dapat memiliki performa yang lebih konsisten pada berbagai kondisi pertandingan.

LAMPIRAN

A. Repository Github: https://github.com/gunturanggoro/Tubes_AyamGaJago

DAFTAR PUSTAKA

- [1] Amazon Web Services, "What is an API (Application Programming Interface)?," aws.amazon.com. [Online]. Available: <https://aws.amazon.com/what-is/api/>. [Accessed: May 23, 2026].
- [2] F. N. Larsen, "Robocode Tank Royale Documentation," robocode-dev.github.io, 2022. [Online]. Available: <https://robocode-dev.github.io/tank-royale/>. [Accessed: May 23, 2026].
- [3] J. Buchea, "Semantic Commit Messages," gist.github.com. [Online]. Available: <https://gist.github.com/joshbucha/6f47e86d2510bce28f8e7f42ae84c716>. [Accessed: May 23, 2026].
- [4] F. N. Larsen, "Robocode Home," robocode.sourceforge.io, 2026. [Online]. Available: <https://robocode.sourceforge.io/>. [Accessed: May 23, 2026].
- [5] Ariel-HS, "Tubes1-IF2211-Starter-Pack, v1.0," GitHub, Mar. 2, 2025. [Online]. Available: <https://github.com/Ariel-HS/tubes1-if2211-starter-pack/releases/tag/v1.0>. [Accessed: May 23, 2026].
- [6] Microsoft, ".NET - Build Modern Apps and Powerful Cloud Services," dotnet.microsoft.com. [Online]. Available: <https://dotnet.microsoft.com/en-us/>. [Accessed: May 23, 2026].
- [7] Microsoft, "C# Guide - .NET Managed Language," learn.microsoft.com. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/>. [Accessed: May 23, 2026].